

Back tracking

branch-and-bound



Algorithm Design





西南财经大学

SOUTHWESTERN UNIVERSITY OF FINANCE AND ECONOMICS

Algorithm Design & Analysis

Lecture 13

SWUFE

经世济民，孜孜以求



回溯算法的设计与实现

搜索树结点数的估计

图的着色问题

回溯算法的递归与迭代实现

回溯算法的基本思想和适用条件

**回溯算法的例子：
n皇后放置、0-1背包、货郎问题**



一、回溯算法例子

4皇后问题

0-1背包问题

货郎问题

适用条件：多米诺性质

二、回溯算法设计步骤

递归实现

迭代实现

装载问题

图着色问题

三、分支限界

一般背包问题、最大团问题

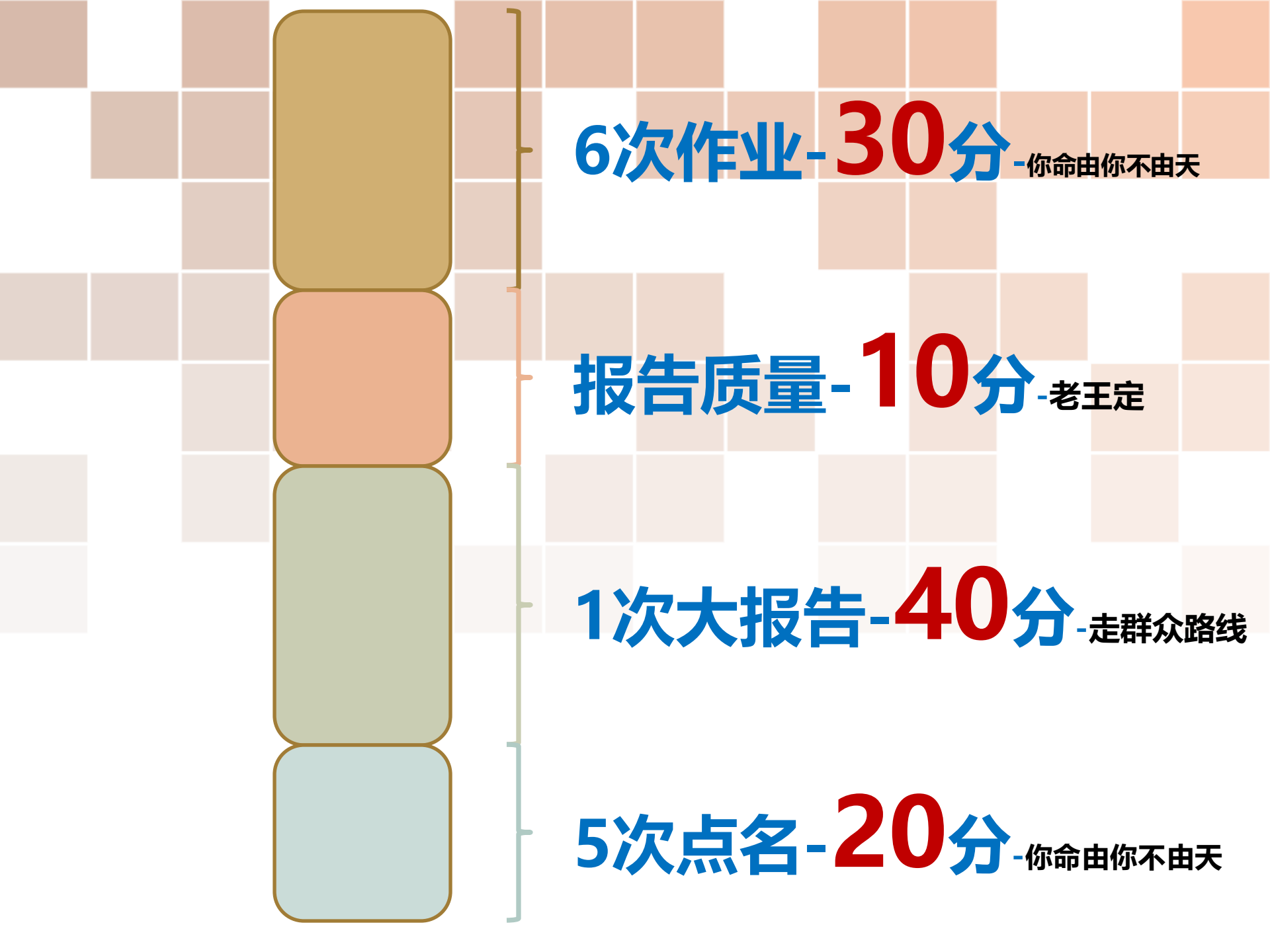
货郎问题、圆排列问题、邮资问题



1.通过分支定界算法解决以下背包问题的实例，背包限重16：

商品	重量	价值
1	10	100
2	7	63
3	8	56
4	4	12

2.设计一个更加复杂的代价函数通过限界方法解决上述问题；



6次作业-30分-你命由你不由天

报告质量-10分-老王定

1次大报告-40分-走群众路线

5次点名-20分-你命由你不由天

● n皇后问题

● 问题定义-1

● 问题定义2

● 4后问题

● 深度宽度遍历

● 搜索空间-4叉树

● 搜索空间-8后

● 8后时间复杂度

● 回溯基本思想

● 小结

n皇后问题-1



马克斯·贝瑟尔 1848年

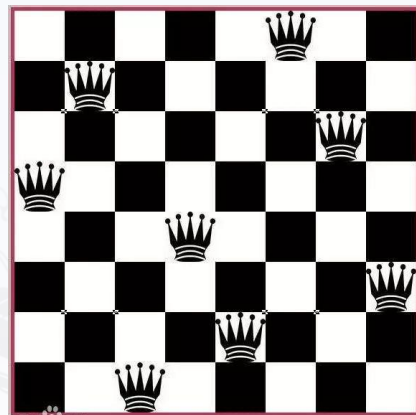
1850年高斯小王子在思考8皇后问题



有多少种摆法

每种怎么摆放

王子认为有76种摆法



- 1854年柏林《象棋杂志》发表了40种摆法
 - 现代计算机精确计算出了92种摆法



n皇后问题

问题定义-1

问题定义2

4后问题

深度宽度遍历

搜索空间-4叉树

搜索空间-8后

8后时间复杂度

回溯基本思想

小结

n皇后问题-2

2格棋盘

1	
2	

1	
	2

	1
2	

	1
	2

3格棋盘

固定前两个棋子有多少种摆法？

1		
2		
3		

1		
2		
	3	

1		
2		
		3

移动2号棋子呢？

移动1号棋子呢？

.....

4格棋盘解空间（摆法）呢？



● n皇后问题

● 问题定义-1

● 问题定义2

● 4后问题

● 深度宽度遍历

● 搜索空间-4叉树

● 搜索空间-8后

● 8后时间复杂度

● 回溯基本思想

● 小结

n皇后问题-4后

4后问题：在 4×4 方格棋盘上放置4个皇后，使得没有两个皇后在同一行、同一列、也不在同一条45度的斜线上。

问有多少种可能的布局？

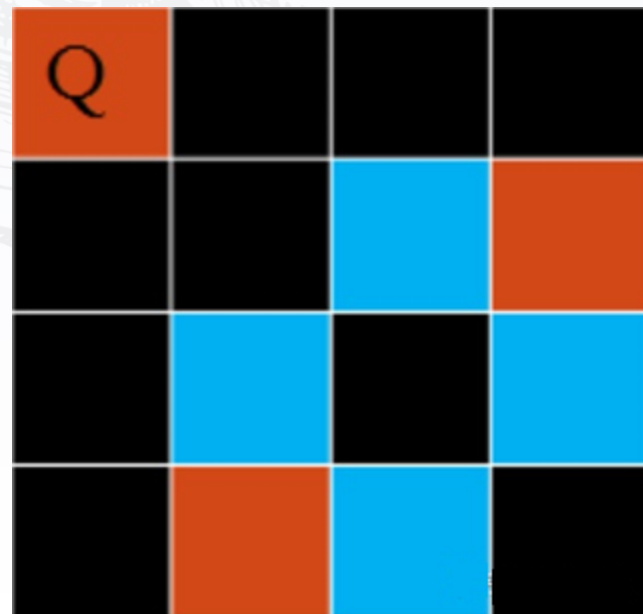
解是4维向量 $\langle x_1, x_2, x_3, x_4 \rangle$

解： $\langle 2, 4, 1, 3 \rangle, \langle 3, 1, 4, 2 \rangle$

推广到8后问题

解：8维向量问题，有92个。

例如： $\langle 1, 5, 8, 6, 3, 7, 2, 4 \rangle$ 是解



4格棋盘解是如何得到呢？



● n皇后问题

● 问题定义-1

● 问题定义2

● 4后问题

● 深度宽度遍历

● 搜索空间-4叉树

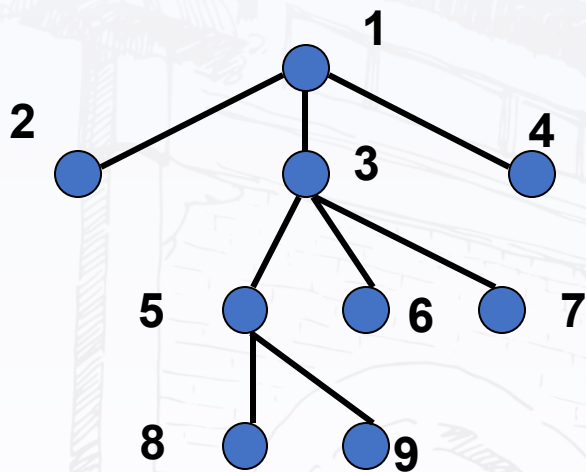
● 搜索空间-8后

● 8后时间复杂度

● 回溯基本思想

● 小结

先回顾深度与宽度优先搜索



深度优先访问顺序

1, 2, 3, 5, 8, 9, 6, 7, 4

宽度优先访问顺序

1, 2, 3, 4, 5, 6, 7, 8, 9

● n皇后问题

● 问题定义-1

● 问题定义2

● 4后问题

● 深度宽度遍历

● 搜索空间-4叉树

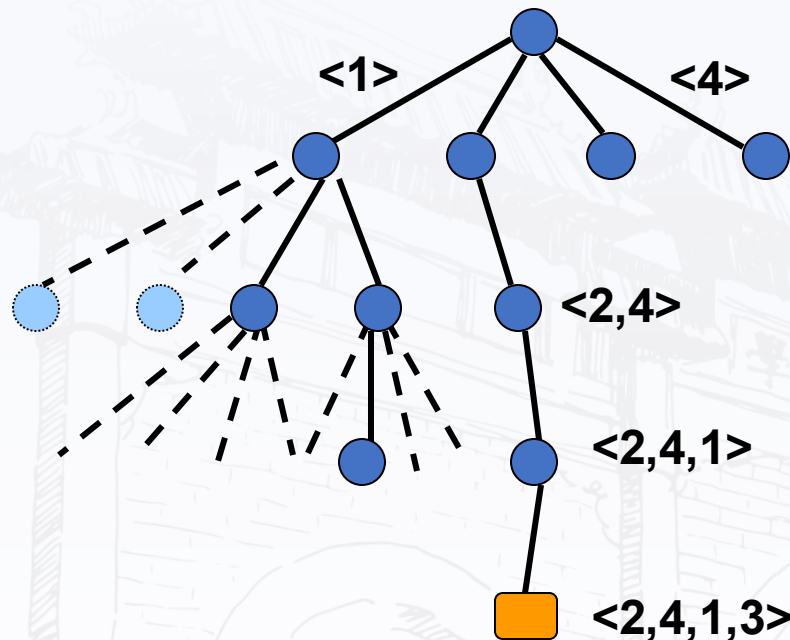
● 搜索空间-8后

● 8后时间复杂度

● 回溯基本思想

● 小结

搜索空间：引入4叉树



1. 每个结点有4个儿子，分别代表选1, 2, 3, 4列为位置
2. 第*i*层选择解向量中第*i*个分量的值最深层的树叶是解
3. 按深度优先次序遍历树，找到所有解



● n皇后问题

● 问题定义-1

● 问题定义2

● 4后问题

● 深度宽度遍历

● 搜索空间-4叉树

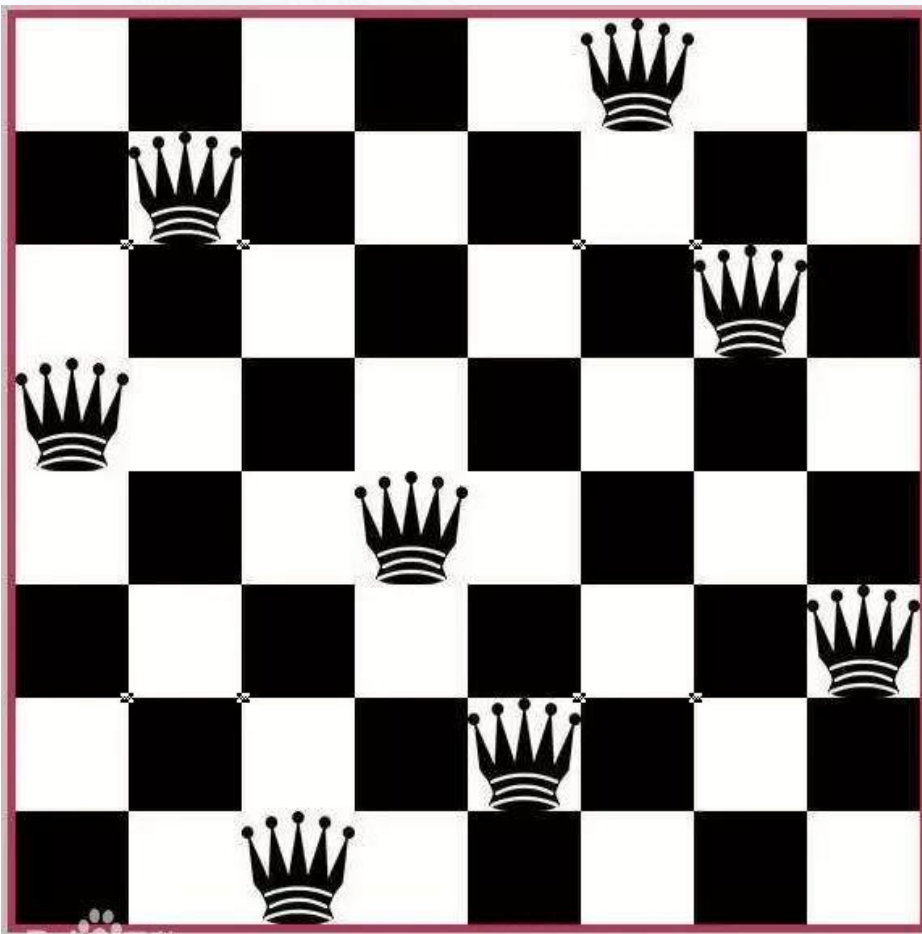
● 搜索空间-8后

● 8后时间复杂度

● 回溯基本思想

● 小结

搜索空间：8皇后问题



8格棋盘解如何得到?

时间复杂度大小?

● n皇后问题

● 问题定义-1

● 问题定义2

● 4后问题

● 深度宽度遍历

● 搜索空间-4叉树

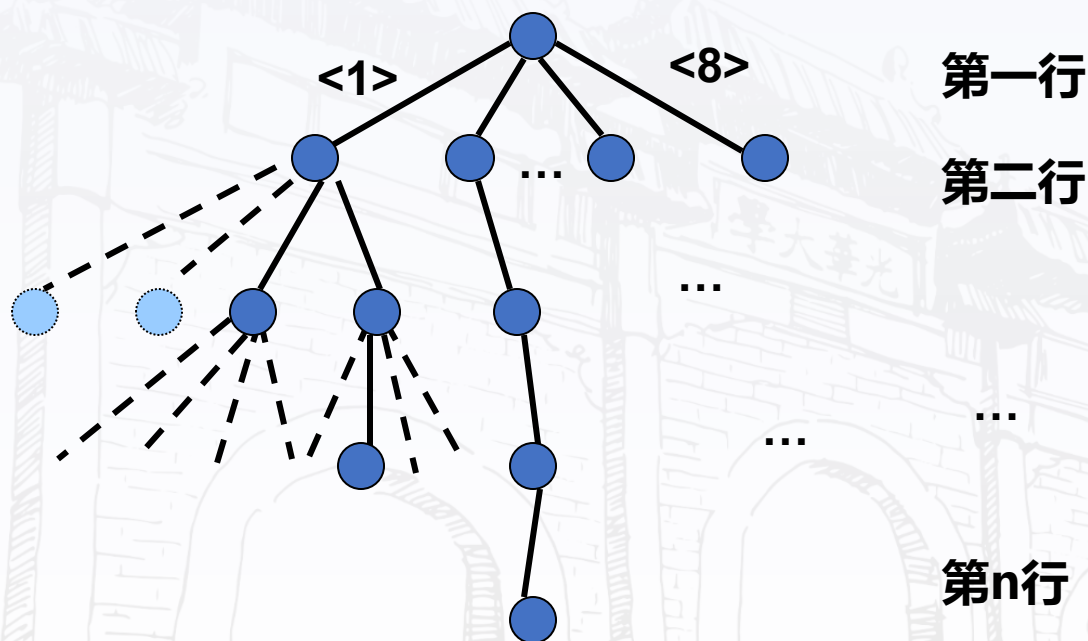
● 搜索空间-8后

● 8后时间复杂度

● 回溯基本思想

● 小结

n皇后问题：时间复杂度



时间复杂度如何考虑？结点数？皇后影响哪几个方向？



n皇后问题

问题定义-1

问题定义2

4后问题

深度宽度遍历

搜索空间-4叉树

搜索空间-8后

8后时间复杂度

回溯基本思想

小结

回溯的基本思想

适用问题

1

求解搜索问题和优化问题

搜索空间

2

树

结点对应部分解向量

树叶对应可行解

搜索过程

3

采用系统的方法隐含遍历

搜索树

搜索策略

4

深度优先

宽度优先

函数优先

宽深结合等

存储

7

当前路径

结点分支判定条件

5

满足

约束
条件

分支扩张解向量

不满足

回溯到父结点

结点状态动态生成

6

白结点-尚未访问

灰结点-正在访问该结点为根的子树

黑节点-该结点为根的子树遍历完成



一、回溯算法例子

4皇后问题

0-1背包问题

货郎问题

适用条件：多米诺性质

二、回溯算法设计步骤

递归实现

迭代实现

装载问题

图着色问题

三、分支限界

一般背包问题、最大团问题

货郎问题、圆排列问题、邮资问题



0-1背包问题

问题定义

算法设计

回溯实例-1

回溯实例-2

时间复杂度

一般背包问题

0-1背包问题

问题：

有 n 种物品，每种物品只有1个.第 i 种物品价值为 v_i ，重量为 w_i ， $i = 1, 2 \dots, n$.问如何选择放入背包的物品，使得总重量不超过 B ，而价值达到最大？

实例：

$$V = \{12, 11, 9, 8\}, W = \{8, 6, 4, 3\}, B = 13$$

最优解：

$$\langle 0, 1, 1, 1 \rangle, \text{价值: } 28, \text{重量: } 13$$



0-1背包问题

问题定义

算法设计

回溯实例-1

回溯实例-2

时间复杂度

一般背包问题

算法设计

解： n 维0-1向量 $\langle x_1, x_2, \dots, x_n \rangle, x_i = 1 \Leftrightarrow$ 物品 i 选入背包

结点： $\langle x_1, x_2, \dots, x_n \rangle$ (部分向量)

搜索空间： 0-1取值二叉树，称为子集树，有 2^n 片树叶。

可行解： 满足约束条件 $\sum_{i=1}^n w_i x_i \leq B$ 的解

最优解： 可行解中价值最大



0-1背包问题

问题定义

算法设计

回溯实例-1

回溯实例-2

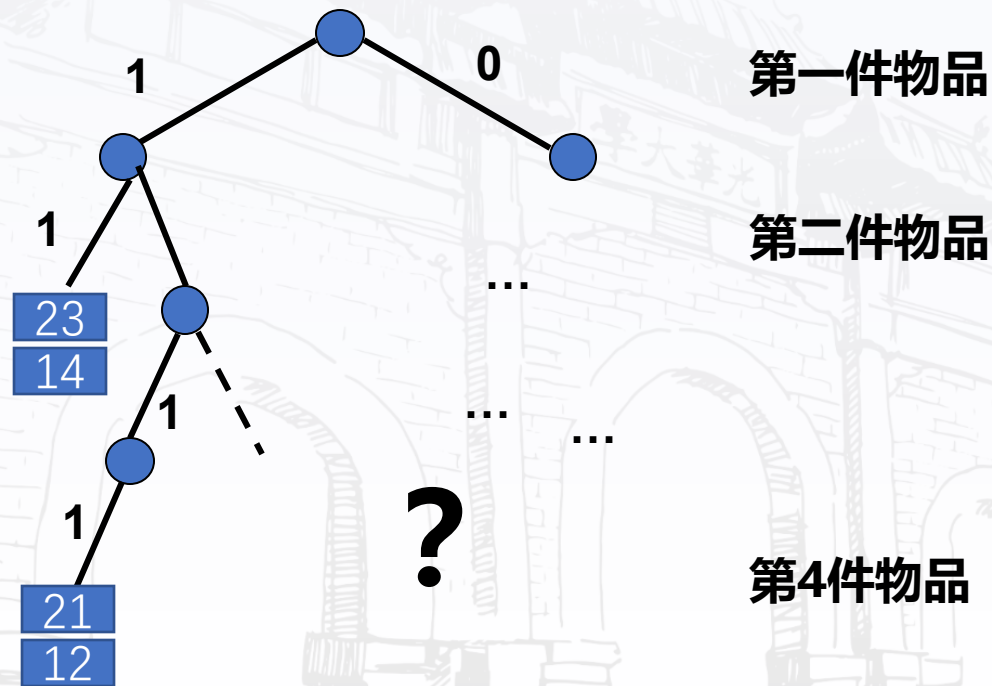
时间复杂度

一般背包问题

0-1背包回溯实例-1

实例：

$$V=\{12,11,9,8\}, W=\{8,6,4,3\}, B=13$$



0-1背包问题

问题定义

算法设计

回溯实例-1

回溯实例-2

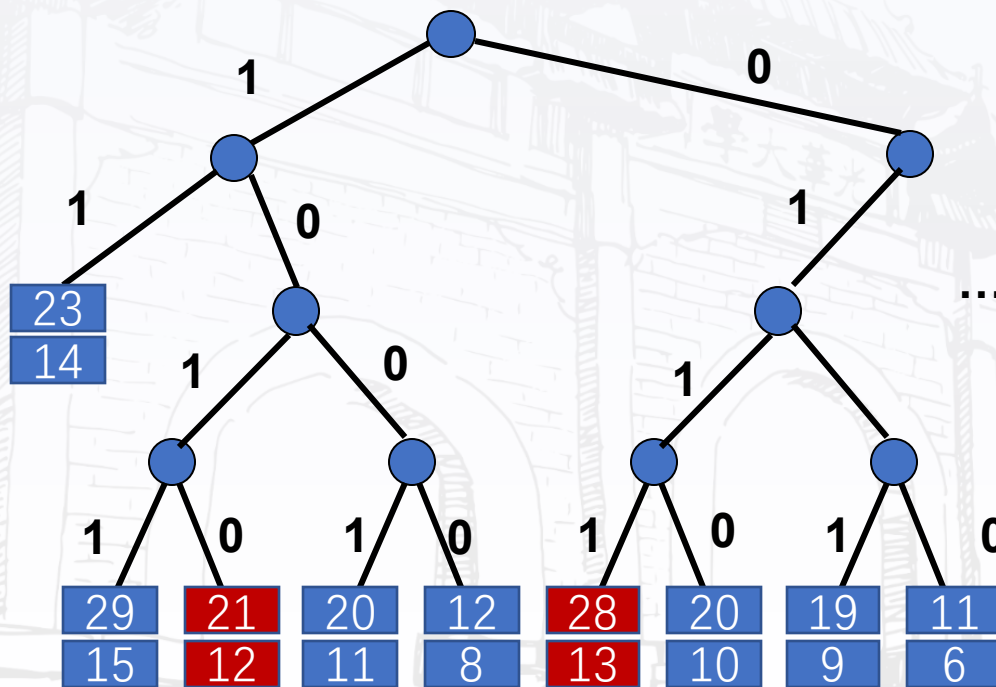
时间复杂度

一般背包问题

0-1背包回溯实例-2

实例：

$$V=\{12,11,9,8\}, W=\{8,6,4,3\}, B=13$$



第一件物品

第二件物品

第4件物品

$\langle 0,1,1,1 \rangle$ 可行解: $x_1=0, x_2=1, x_3=1, x_4=1$. 价值:28, 重量:13

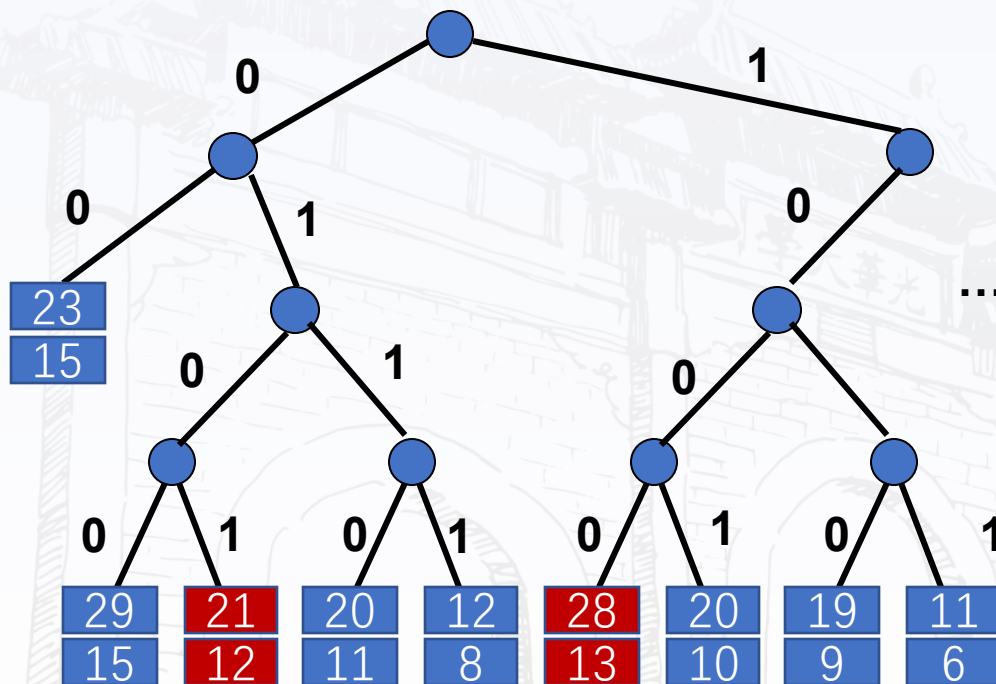
$\langle 1,0,1,0 \rangle$ 可行解: $x_1=1, x_2=0, x_3=1, x_4=0$. 价值:21, 重量:12



0-1背包问题

- 问题定义
- 算法设计
- 回溯实例-1
- 回溯实例-2
- 时间复杂度
- 一般背包问题

0-1背包回溯时间复杂度



时间复杂度如何考虑？结点数？有哪些额外计算？

$$1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1 \leq 2 \times 2^n = O(2^n)$$



0-1背包问题

问题定义

算法设计

回溯实例-1

回溯实例-2

时间复杂度

一般背包问题

一般背包问题回溯方法

问题：一个旅行者随身携带一个背包，可以放入背包的物品有 n 种，每种物品的重量和价值分别为 w_i, v_i .如果背包的最大重量限制是 b ，每种物品可以放多个，怎么选择放入背包的物品以使得背包的价值最大？不妨设上述 w_i, v_i, b 都是正整数.

实例：

$$n = 4, b = 10$$

$$v_1 = 1, v_2 = 3, v_3 = 5, v_4 = 9$$

$$w_1 = 2, w_2 = 3, w_3 = 4, w_4 = 7,$$

请用回溯算法求解

一、回溯算法例子

4皇后问题

0-1背包问题

货郎问题

适用条件：多米诺性质

二、回溯算法设计步骤

递归实现

迭代实现

装载问题

图着色问题

三、分支限界

一般背包问题、最大团问题

货郎问题、圆排列问题、邮资问题



● 货郎问题

● 问题定义

● 实例

● 搜索空间-1

● 搜索空间-2

● 回溯小结

货郎问题

问题：有 n 个城市，已知任两个城市之间的距离，求一条每个城市恰好经过1次的回路，使得总长度最小。

建模：城市集合 $C = \{c_1, c_2, \dots, c_n\}$ ，距离 $d(c_i, c_j) \in \mathbb{Z}^+, 1 \leq i \leq j \leq n$

求： $1, 2, \dots, n$ 的排列 $k_1, k_2, \dots, k_n$ 使得

$$\min\{\sum_{i=1}^{n-1} d(c_i, c_{i+1}) + d(c_n, c_{k_1})\}$$

● 货郎问题

● 问题定义

● 实例

● 搜索空间-1

● 搜索空间-2

● 回溯小结

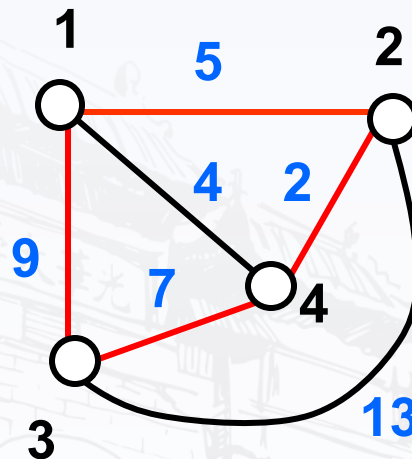
实例

$$C = \{1, 2, 3, 4\}$$

$$d(1, 2) = 5, d(1, 3) = 9$$

$$d(1, 4) = 4, d(2, 3) = 13$$

$$d(2, 4) = 2, d(3, 4) = 7$$



请用回溯算法求解

● 货郎问题

● 问题定义

● 实例

● 搜索空间-1

● 搜索空间-2

● 回溯小结

搜索空间-1

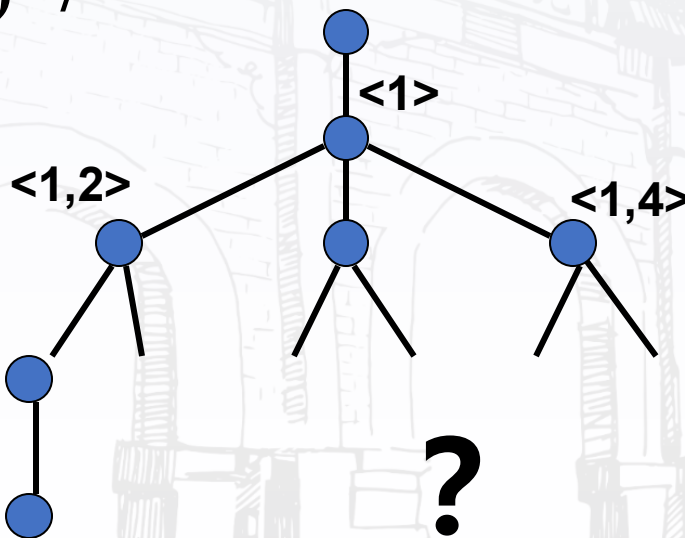
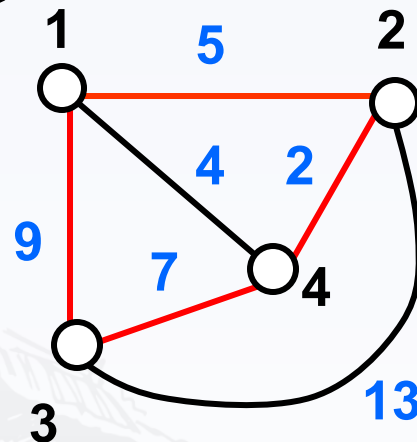
排列树,有 $(n-1)!$ 片树叶

$$C=\{1, 2, 3, 4\}$$

$$d(1,2)=5, d(1,3)=9$$

$$d(1,4)=4, d(2,3)=13$$

$$d(2,4)=2, d(3,4)=7$$



● 货郎问题

● 问题定义

● 实例

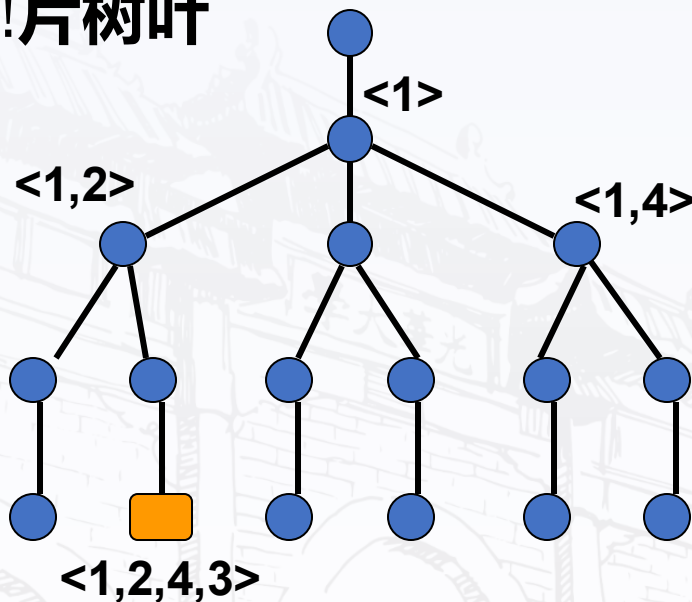
● 搜索空间-1

● 搜索空间-2

● 回溯小结

搜索空间-2

排列树,有 $(n-1)!$ 片树叶



时间复杂度: 结点数*额外开销 $O(1)$

结点数: $1 + 1 + (n-1) + (n-1)(n-2) \dots (n-1)(n-2) + (n-1)! \leq O((n-1)!)$ *运算较复杂*

小结

- **回溯算法的例子：**

n皇后问题， 0-1背包问题， 货郎问题

- **解：** 向量

- **搜索空间：** 树，可能是n叉树、子集树、排列树等等，树的结点对应于部分向量，可行解在叶节点

- **搜索方法：** 深度优先、宽度优先、跳跃式遍历搜索树，找到解

一、回溯算法例子

4皇后问题

0-1背包问题

货郎问题

适用条件：多米诺性质

二、回溯算法设计步骤

递归实现

迭代实现

装载问题

图着色问题

三、分支限界

一般背包问题、最大团问题

货郎问题、圆排列问题、邮资问题



● 适用条件

● 问题分析

● 回顾基本思想

● 适用条件

● 反例

问题分析

问题	解性质	解向量描述	搜索空间	搜索方式	约束条件
N皇后	可行解	$\langle x_1, x_2, \dots, x_n \rangle$ x_n :第 <i>i</i> 行列号	n叉树	深度 宽度	彼此不攻击
0-1背包	最优解	$\langle x_1, x_2, \dots, x_n \rangle$ $x_i = 0, 1$ $x_i = 1$ 选 <i>i</i>	子集树	深度 宽度	背包不超重 量
货郎	最优解	$\langle k_1, k_2, \dots, k_n \rangle$ 1,2,.....n的排列	排列树	深度 宽度	选没有经过 的城市
特点	搜索解	不断扩张向量	树	跳跃式 遍历	约束条件 回溯判定

● 适用条件

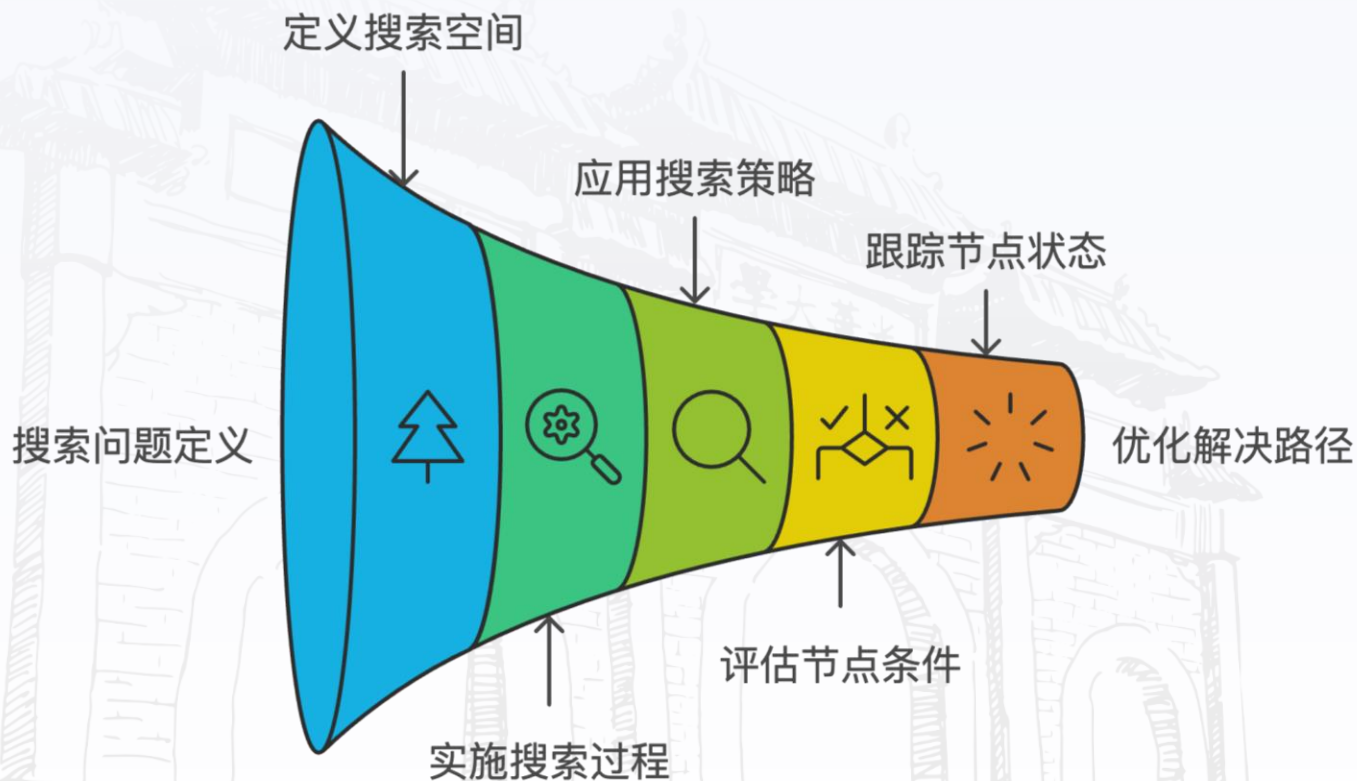
● 问题分析

● 回顾基本思想

● 适用条件

● 反例

回溯算法基本思想





● 适用条件

● 问题分析

● 回顾基本思想

● 适用条件

● 反例

回溯算法适用条件

设 $P(x_1, x_2, \dots, x_i)$ 为真

表示向量 $\langle x_1, x_2, \dots, x_i \rangle$ 满足某个性质

Example: n 后问题中 i 个皇后放置在彼此不能攻击的位置)

多米诺性质

$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

多米诺性质逆否命题

$$\neg P(x_1, x_2, \dots, x_k) \rightarrow \neg P(x_1, x_2, \dots, x_{k+1}) \quad 0 < k < n$$

*** k 维向量不满足约束条件, 扩张向量到 $k+1$ 维仍旧不满足, 可以回溯(及时止损)***



● 适用条件

● 问题分析

● 回顾基本思想

● 适用条件

● 反例

例子

例：求不等式的正数解决

$$5x_1 + 4x_2 - x_3 \leq 10, \quad 1 \leq x_i \leq 3, \quad i=1,2,3$$

$P(x_1, \dots, x_k)$: 意味将 $x_1, x_2, \dots, x_k$ 代入原不等式的相应部分使得左边小于等于10,

不满足多米诺性质：

$$\text{因为： } 5x_1 + 4x_2 - x_3 \leq 10 \not\Rightarrow 5x_1 + 4x_2 \leq 10$$

但是通过变换可以使问题满足多米诺性质：

变换： 令 $x_3 = 3 - x_3'$,

$$5x_1 + 4x_2 + x_3' \leq 13, \quad 1 \leq x_1, x_2 \leq 3, \quad 0 \leq x_3' \leq 2$$



小结

• 回溯算法适用条件：

多米诺性质

• 回溯算法设计步骤：

(1) 定义搜索问题的解向量和每个分量的取值范围

解向量 $\langle x_1, x_2, \dots, x_i \rangle$, 以及 x_i 取值范围

(2) 当 $x_1, x_2, \dots, x_{k-1}$ 确定以后计算 x_k 取值集合 $S_k, S_k \subseteq X_k$

(3) 确定结点儿子的排列规则

(4) 判断是否满足多米诺性质

(5) 搜索策略----深度优先、宽度优先等

(6) 确定每个结点分支约束条件

(7) 确定存储搜索路径的数据结构



一、回溯算法例子

4皇后问题

0-1背包问题

货郎问题

适用条件：多米诺性质

二、回溯算法设计步骤

递归实现

迭代实现

装载问题

图着色问题

三、分支限界

一般背包问题、最大团问题

货郎问题、圆排列问题、邮资问题



回溯设计步骤

递归实现

迭代实现

装载问题

求解思路

伪码

子过程

实例

小结

回溯算法的设计步骤-递归实现

算法 ReBack(k)

1. if $k > n$ then $\langle x_1, x_2, \dots, x_n \rangle$ 是解 // 若大于 n 则说明各个分量已得到解
2. else while $S_k \neq \emptyset$ do
3. $x_k \leftarrow S_k$ 中最小值
4. $S_k \leftarrow S_k - \{x_k\}$
5. 计算 S_{k+1} // 计算下一层的取值范围
6. ReBack($k+1$) // 调用下一层

算法 递归回溯 ReBacktrack(n)

输入: n

输出: 所有的解

1. for $k \leftarrow 1$ to n 计算 X_k 且 $S_k \leftarrow X_k$ // 计算初值, 部分 X 通过限制条件可约束
2. ReBack(1)



回溯设计步骤

- 递归实现
- 迭代实现
- 装载问题
- 求解思路
- 伪码
- 子过程
- 实例
- 小结

回溯算法的设计步骤-迭代实现

算法 Backtrack(n)

输入: n

输出: 所有的解

1. 对于 $i=1, 2, \dots, n$ 确定 X_i //确定初值
2. $k \leftarrow 1$
3. 计算 S_k
4. while $S_k \neq \emptyset$ do //满足约束条件, 分支搜索
5. $x_k \leftarrow S_k$ 中最小值; $S_k \leftarrow S_k - \{x_k\}$
6. if $k < n$ then
7. $k \leftarrow k+1$; 计算 S_k
8. else $\langle x_1, x_2, \dots, x_n \rangle$ 是解
9. if $k > 1$ then $k \leftarrow k-1$; goto 4 //回溯

回溯设计步骤

- 递归实现
- 迭代实现
- 装载问题
- 求解思路
- 伪码
- 子过程
- 实例
- 小结

装载问题

问题 n 个集装箱装上2艘载重分别为 c_1 和 c_2 的轮船, w_i 为集装箱 i 的重量, 且

$$\sum_{i=1}^n w_i \leq c_1 + c_2$$

问是否存在一种合理的装载方案将 n 个集装箱装上轮船? 如果有, 给出一种方案.

实例:

$$W = \langle 90, 80, 40, 30, 20, 12, 10 \rangle$$

$$c_1 = 152, c_2 = 130$$

解: 1, 3, 6, 7装第一艘船, 其余第2艘船

回溯设计步骤

- 递归实现
- 迭代实现
- 装载问题
- 求解思路**
- 伪码
- 子过程
- 实例
- 小结

求解思路

输入：集装箱重量 $W = \langle w_1, w_2, \dots, w_n \rangle$, c_1 和 c_2 是船的最大载重

算法思想：

令第一条船的装入量为 W_1

1. 用回溯算法求使得 $c_1 - W_1$ 达到最小的装载方案.

2. 若满足

$$w_1 + w_2 + \dots + w_n - W_1 \leq c_2$$

则回答 “Yes”



, 否则回答 “No”





回溯设计步骤

- 递归实现
- 迭代实现
- 装载问题
- 求解思路
- 伪码
- 子过程
- 实例
- 小结

伪码

算法 Loading (W, c_1),

1. Sort(W); //对 $w_1, w_2, \dots, w_n$ 按照从大到小排序
2. $B \leftarrow c_1$; $best \leftarrow c_1$; $i \leftarrow 1$; // $best$ 是装船空隙
3. while $i \leq n$ do
4. if 装入 i 后重量不超过 c_1
5. then $B \leftarrow B - w_i$; $x[i] \leftarrow 1$; $i \leftarrow i + 1$;
6. else $x[i] \leftarrow 0$; $i \leftarrow i + 1$;
7. if $B < best$ then 记录解; $best \leftarrow B$;
8. Backtrack(i); //回溯到父节点
9. if $i = 1$ then return 最优解
10. else goto 3.



回溯设计步骤

- 递归实现
- 迭代实现
- 装载问题
- 求解思路
- 伪码
- 子过程
- 实例
- 小结

子过程Backtrack

算法 Backtrack(i)

1. while $i > 1$ and $x[i] = 0$ do //回溯
2. $i \leftarrow i - 1$;
3. if $x[i] = 1$ //沿着右分支一直回溯发现左分支边，或到根为止
4. then $x[i] \leftarrow 0$; //转到右子树
5. $B \leftarrow B + w_i$;
6. $i \leftarrow i + 1$



小结

回溯算法的实现：递归实现、迭代实现

装载问题：

问题描述

算法伪码

最坏情况下时间复杂度 $O(2^n)$

一、回溯算法例子

4皇后问题

0-1背包问题

货郎问题

适用条件：多米诺性质

二、回溯算法设计步骤

递归实现

迭代实现

装载问题

图着色问题

三、分支限界

一般背包问题、最大团问题

货郎问题、圆排列问题、邮资问题



图的着色问题

问题定义

实例

算法设计

解向量

运行实例-1

运行实例-2

运行实例-其他情况

改进方法

着色问题应用

小结

图的着色问题

输入： 给定无向连通图 G 和 m 种颜色，用这些颜色给图的顶点着色，每个顶点一种颜色。

要求是： G 的每条边的两个顶点着不同颜色。

输出： 给出所有可能的着色方案；如果不存在着这样的方案，则回答 “No”。

图的着色问题

实例

问题定义

实例

算法设计

解向量

运行实例-1

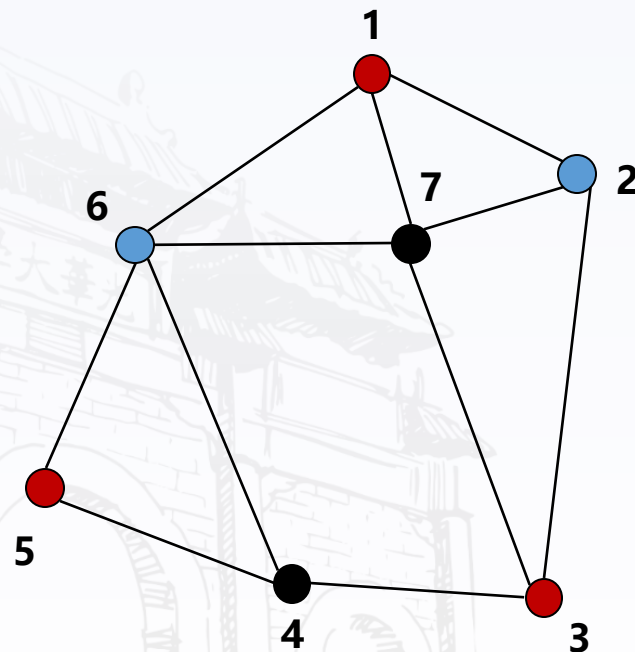
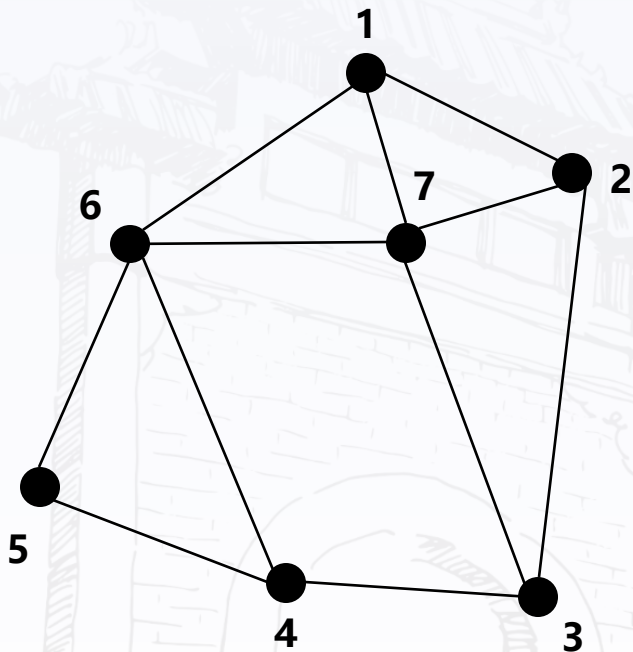
运行实例-2

运行实例-其他情况

改进方法

着色问题应用

小结



$n=7$ 个顶点, $m=3$ 种颜色



图的着色问题

算法设计

搜索树： m 叉树

约束条件： 在结点 $\langle x_1, x_2, \dots, x_n \rangle$ 处顶点 $n+1$ 的邻接表中已经用过的颜色不能再用

回溯： 如果邻接表中的结点已用过 m 中颜色， $n+1$ 结点没法着色，则回溯到其父节点。

搜索策略： 深度优先

时间复杂度： $O(nm^n)$

● 问题定义

● 实例

● 算法设计

● 解向量

● 运行实例-1

● 运行实例-2

● 运行实例-其他情况

● 改进方法

● 着色问题应用

● 小结



图的着色问题

问题定义

实例

算法设计

解向量

运行实例-1

运行实例-2

运行实例-其他情况

改进方法

着色问题应用

小结

解向量

设:

$$G = (V, E), V = \{1, 2, \dots, n\},$$

颜色编号: $1, 2, \dots, m$

解向量: $\langle x_1, x_2, \dots, x_n \rangle,$

$$x_1, x_2, \dots, x_n \in \{1, 2, \dots, m\}$$

结点的部分向量 $\langle x_1, x_2, \dots, x_n \rangle, 1 \leq k \leq n,$

表示只给顶点 $1, 2, \dots, k$ 着色的部分方案



图的着色问题

运行实例-1

问题定义

实例

算法设计

解向量

运行实例-1

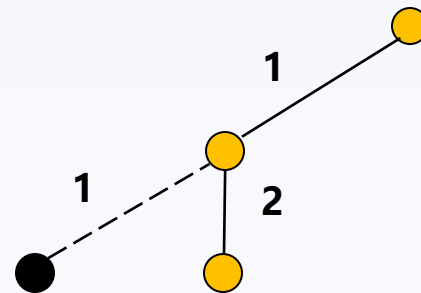
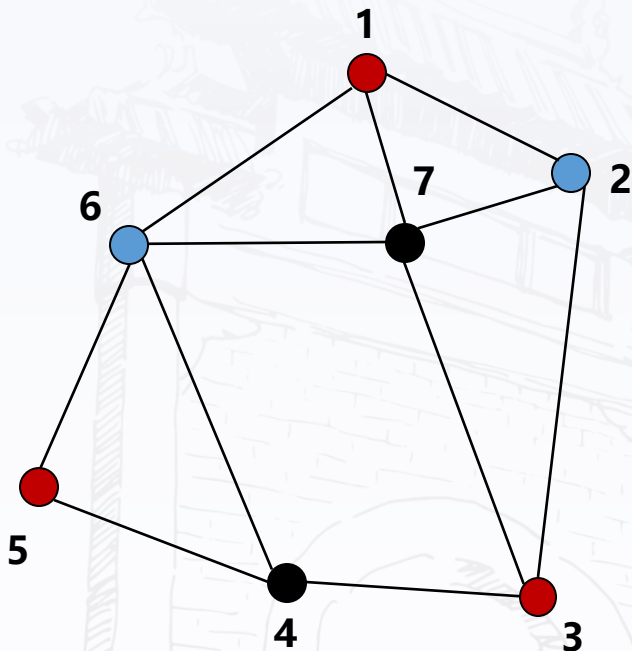
运行实例-2

运行实例-其他情况

改进方法

着色问题应用

小结



结点1

结点2

结点3

结点4

结点5

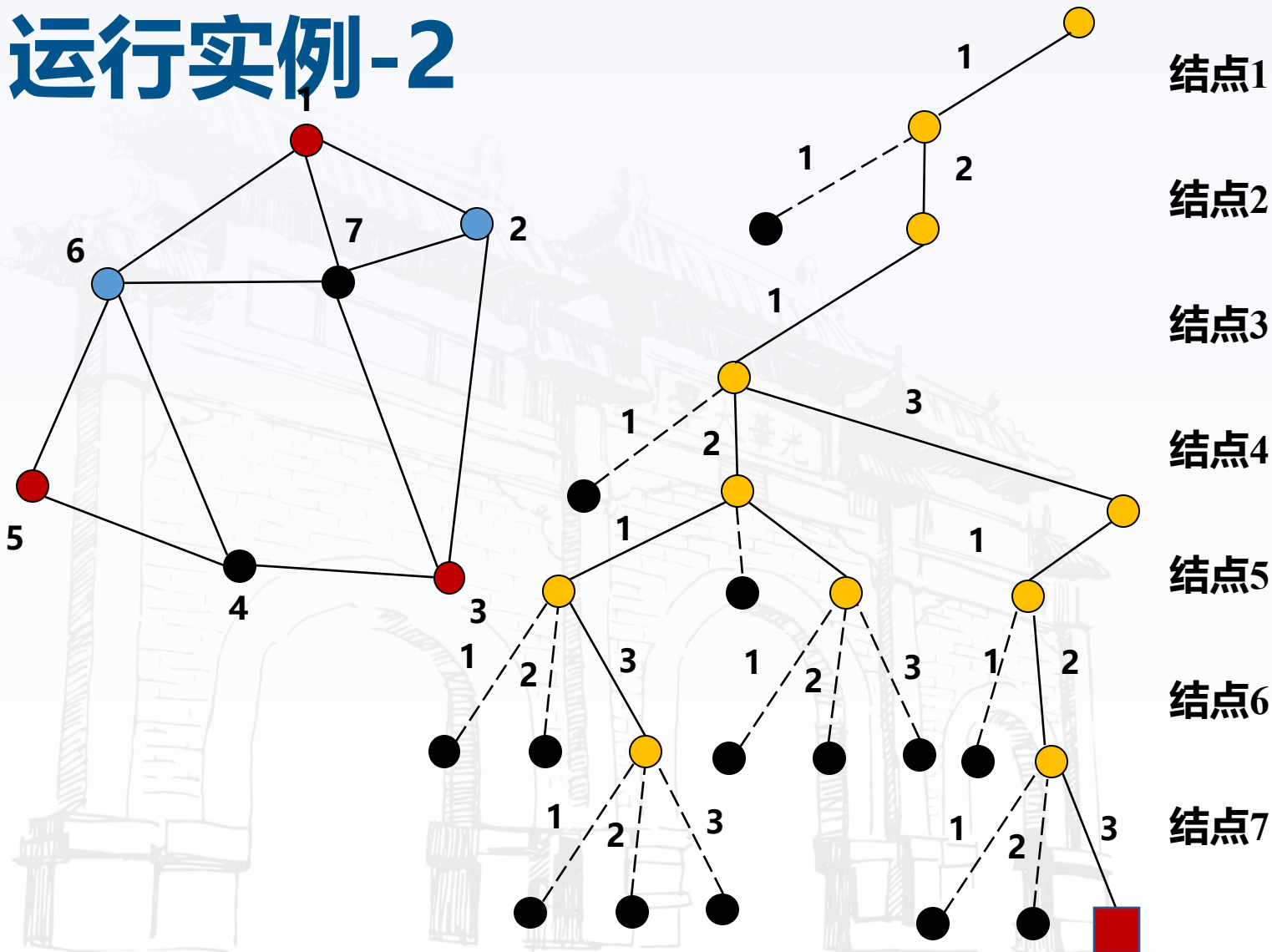
结点6

结点7

?

图的着色问题

运行实例-2



结点1

结点2

结点3

结点4

结点5

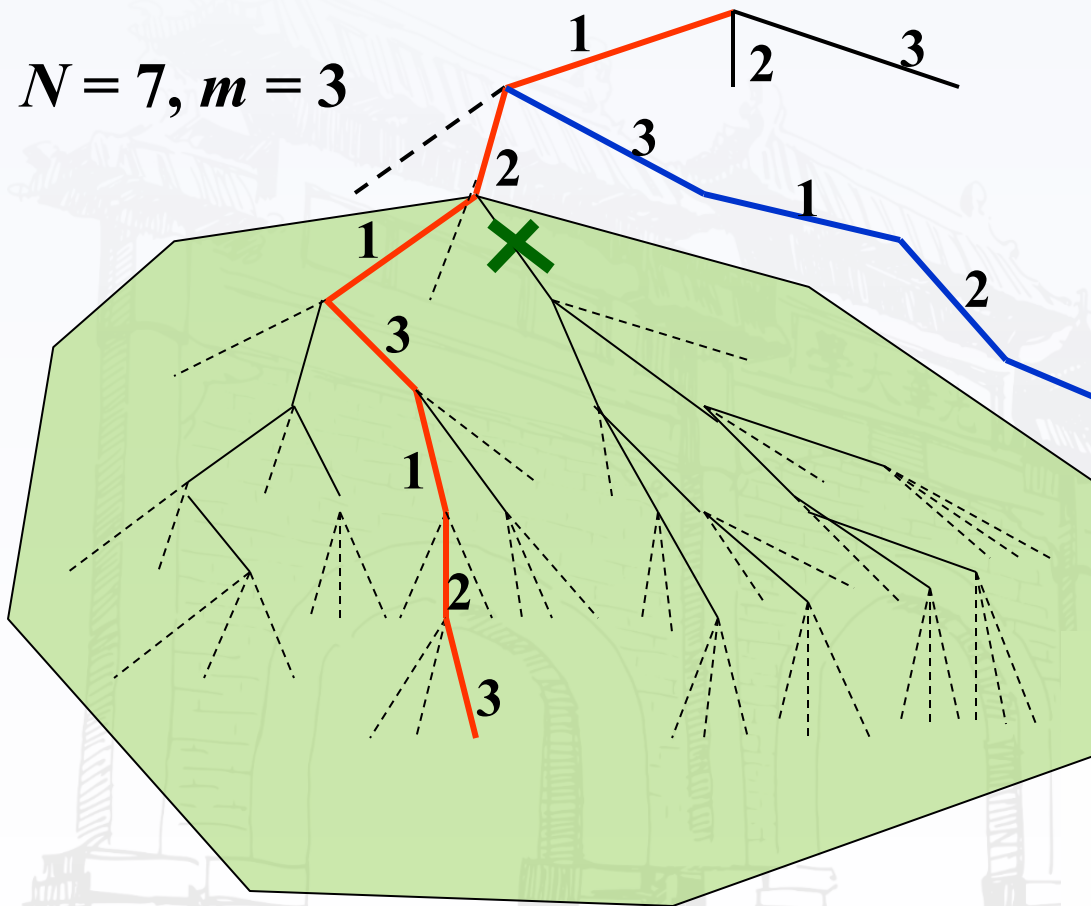
结点6

结点7

图的着色问题

实例-其他搜索情况

$N = 7, m = 3$



$\langle 1 \rangle$

$\langle 1, 2 \rangle$

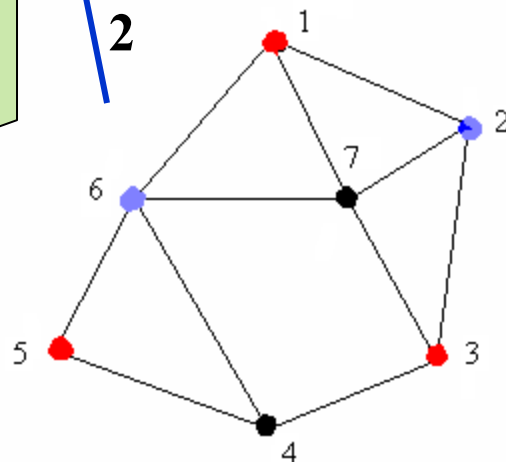
$\langle 1, 2, 1 \rangle$

$\langle 1, 2, 1, 3 \rangle$

$\langle 1, 2, 1, 3, 1 \rangle$

$\langle 1, 2, 1, 3, 1, 2 \rangle$

$\langle 1, 2, 1, 3, 1, 2, 3 \rangle$



第一个解向量: $\langle 1, 2, 1, 3, 1, 2, 3 \rangle$

图的着色问题

时间复杂度与改进途径

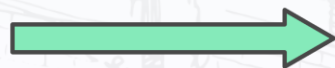
时间复杂度: $O(nm^n)$

如何高效搜索解空间?



≠ 利用对称性

利用对称性将搜索空间减少到 1/3, 提高效率。



检查顶点邻接性

验证是否有与选定节点相邻的顶点, 以确定是否存在解。



回溯

如果存在相邻顶点, 回溯以避免不必要的子树搜索。



图的着色问题

着色问题的应用

- 问题定义
- 实例
- 算法设计
- 解向量
- 运行实例-1
- 运行实例-2
- 运行实例-其他情况
- 改进方法
- 着色问题应用
- 小结



小结

回溯算法的实现：递归实现、迭代实现

装载问题：

问题描述

算法伪码

最坏情况下时间复杂度 $O(2^n)$

图的着色问题

问题描述、实例、改进思路



一、回溯算法例子

4皇后问题

0-1背包问题

货郎问题

适用条件：多米诺性质

二、回溯算法设计步骤

递归实现

迭代实现

装载问题

图着色问题

三、分支限界

一般背包问题、最大团问题

货郎问题、圆排列问题、邮资问题



回溯算法的应用与小结

回溯算法小结

最大团
问题

货郎
问题

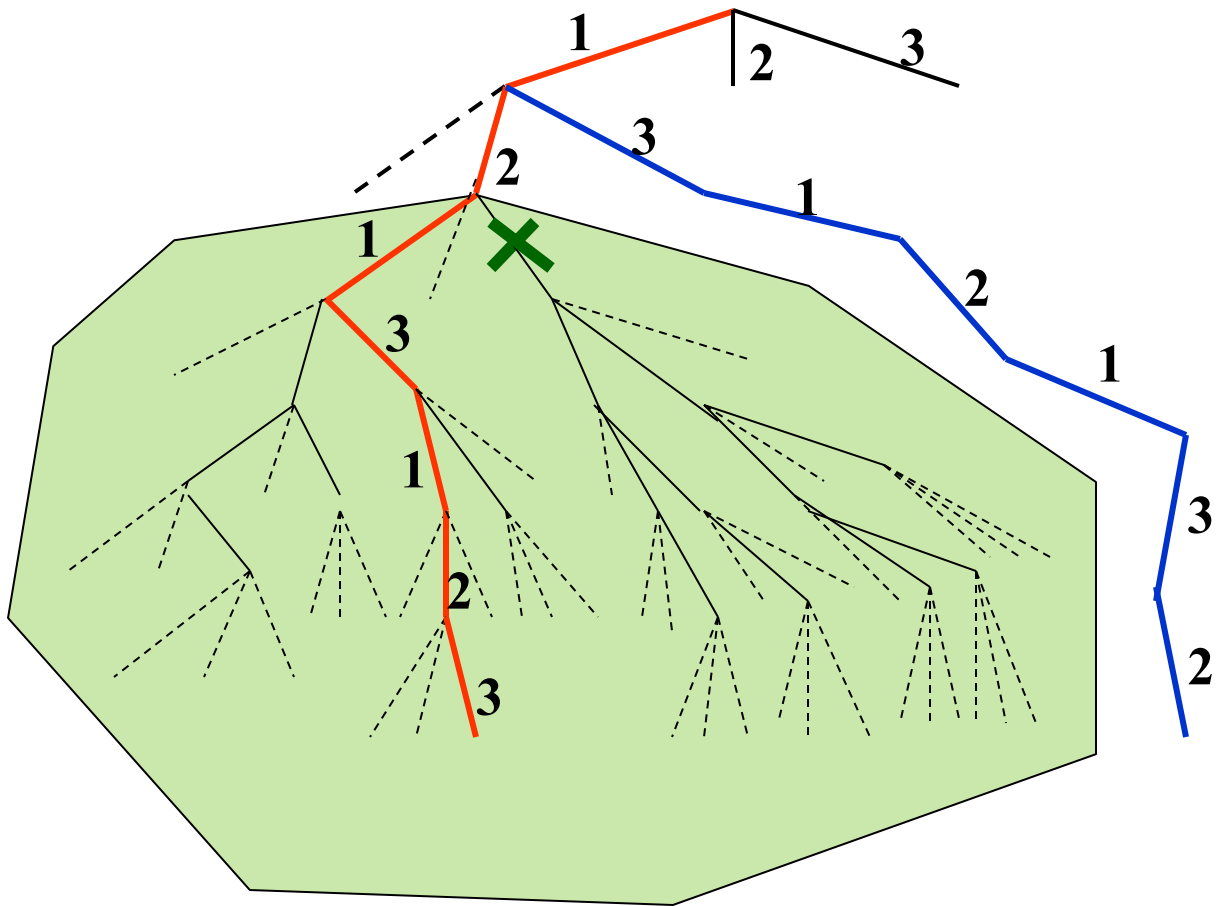
圆排列
问题

连续邮
资问题

分支限界：背包问题



为何需要分支限界?



第一个解向量: $\langle 1, 2, 1, 3, 1, 2, 3 \rangle$

时间复杂度: $O(nm^n)$



Example Assignment Problem

Person	Job1	Job2	Job3	Job4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

Target = min(cost)

原始问题回溯处理方案？

如何修剪树使搜索空间变小？



● 分支限界

● 分支限界

● 组合优化

● 代价函数

● 界

● 使用分支限界

● 实例分析

● 代价函数设定

● 变元排序实例

● 代价函数与分支策略

● 运行实例

● 小结

分支限界是回溯的变种，用于解决组合优化问题

为何需要分支限界？

回溯空间大

增加约束条件

降低搜索空间

回溯算法慢

裁剪搜索分支

提高回溯速度



● 分支限界

组合优化问题

目标函数：  极大化  极小化

约束条件： 解满足的条件

可行解： 搜索空间满足约束条件的解

最优解： 使得目标函数达到极大（或极小）的可行解

实例： 背包问题

$$\max x_1 + 3x_2 + 5x_3 + 9x_4$$

$$2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10$$

$$x_i \in N, i = 1, 2, 3, 4$$

目标函数

} 约束条件

● 分支限界

● 分支限界

● 组合优化

● 代价函数

● 界

● 使用分支限界

● 实例分析

● 代价函数设定

● 变元排序实例

● 代价函数与分支策略

● 运行实例

● 小结

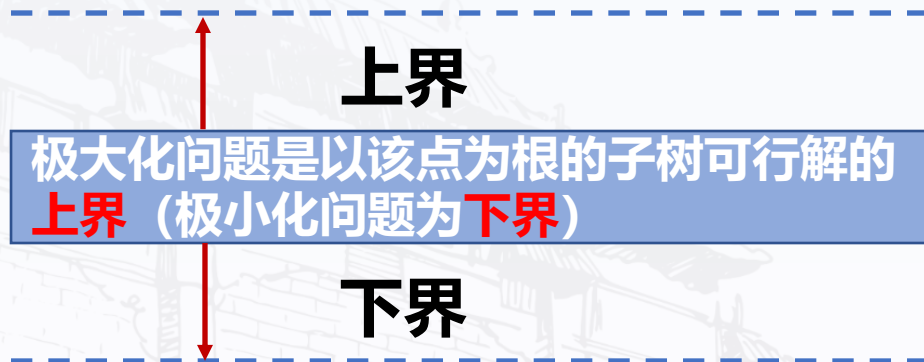
代价函数用于增加分支限界的约束条件

计算位置:

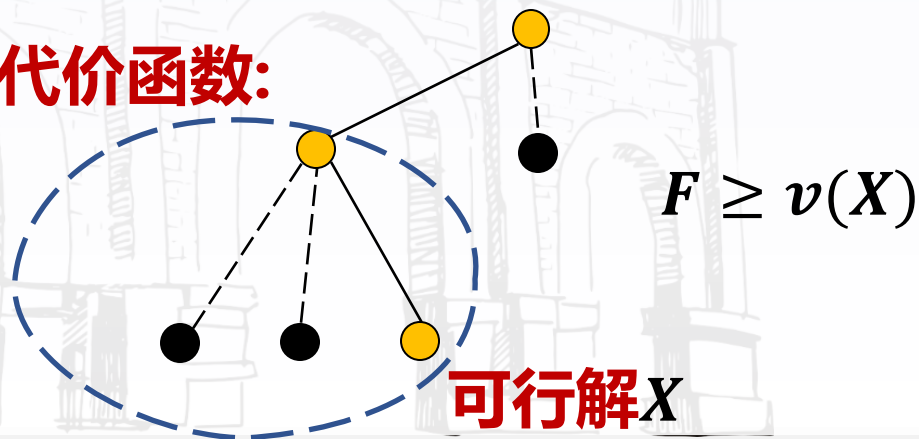
搜索树的结点

代价函数:

极大化问题是以该点为根的子树可行解的
上界 (极小化问题为下界)



F 代价函数:



可行解 X

$$F \geq v(X)$$

● 分支限界

● 分支限界

● 组合优化

● 代价函数

● 界

● 使用分支限界

● 实例分析

● 代价函数设定

● 变元排序实例

● 代价函数与分支策略

● 运行实例

● 小结

界搜索到当前结点时得到的可行解的最大值

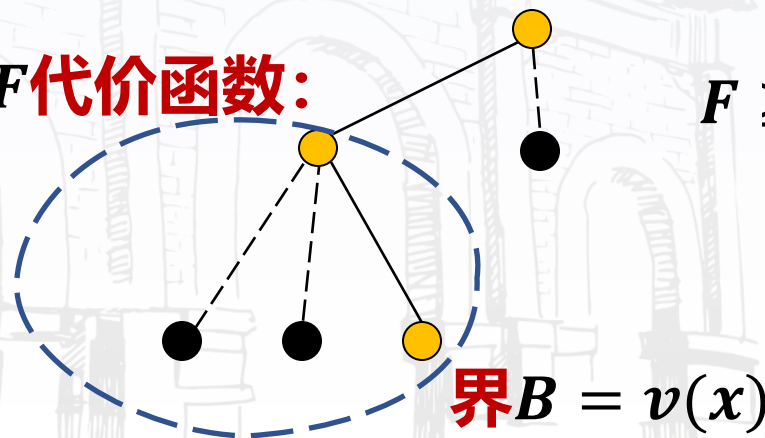
含义：当前得到可行解的目标函数的最大值（极小化问题相反）

初值：极大化问题初值为0（极小化问题为**最大值**）

更新：得到更好的可行解时

F 代价函数：

$$F \geq v(X)$$



$$\text{界 } B = v(x)$$



● 分支限界

使用分支限界

⊘ STOP 停止分支回溯父节点的依据

1. 不满足约束条件
2. 对于极大化问题，代价函数值小于当前界（对于极小化问题是大于界）

✈ UPDATE 界的更新

1. 对于极大化问题，如果一个新的可行解的优化函数值大于（极小化问题为小于）当前的界，则把界更新为该可行的解



分支限界

分支限界

组合优化

代价函数

界

使用分支限界

实例分析

代价函数设定

变元排序实例

代价函数与分支策略

运行实例

小结

实例分析

背包问题

4种物品，重量 w_i 与价值 v_i 分别为

$$v_1 = 1, v_2 = 3, v_3 = 5, v_4 = 9$$

$$w_1 = 2, w_2 = 3, w_3 = 4, w_4 = 7$$

背包重量限制为10

$$\max x_1 + 3x_2 + 5x_3 + 9x_4$$

$$2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10$$

$$x_i \in N, i = 1, 2, 3, 4$$

?

该问题如何设置代价函数?

● 分支限界

● 分支限界

● 组合优化

● 代价函数

● 界

● 使用分支限界

● 实例分析

● 代价函数设定

● 变元排序实例

● 代价函数与分支策略

● 运行实例

● 小结

代价函数的设定

- **目标**: 对结点 $\langle x_1, x_2, \dots, x_k \rangle$, 估计以该节点为根的子树中可行解的上界. 如何估计上界?
- **排序**: 按 v_i/w_i 从大到小排序, $i = 1, 2, \dots, n$

• **代价函数** = 已装入价值 + Δ

Δ : 实际是指还可以继续装入最大价值的上界

$\Delta =$ **背包剩余重量** $\times v_{k+1}/w_{k+1}$ (可装)

$\Delta = 0$ (不可装)





● 分支限界

● 分支限界

● 组合优化

● 代价函数

● 界

● 使用分支限界

● 实例分析

● 代价函数设定

● 变元排序实例

● 代价函数与分支策略

● 运行实例

● 小结

变元排序实例

$$\max x_1 + 3x_2 + 5x_3 + 9x_4$$

$$2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10$$

$$x_i \in N, i = 1, 2, 3, 4$$

- 第一步：对变元重新排序使得 $\frac{v_i}{w_i} \geq \frac{v_{i+1}}{w_{i+1}}$
- 第二步：排序后实例

$$\max 9x_1 + 5x_2 + 3x_3 + x_4$$

$$7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10$$

$$x_i \in N, i = 1, 2, 3, 4$$



● 分支限界

代价函数与分支策略

结点 $\langle x_1, x_2, \dots, x_k \rangle$ 的**代价函数**:

$$\sum_{i=1}^k v_i x_i + \left(b - \sum_{i=1}^k w_i x_i \right) \frac{v_{k+1}}{w_{k+1}}$$

若物品 j 可装入: **即** 某个 $j > k$ 有 $b - \sum_{i=1}^k w_i x_i \geq w_j$

否则代价函数则只有前项 $\sum_{i=1}^k v_i x_i$

分支策略-----**深度优先**

● 分支限界

● 组合优化

● 代价函数

● 界

● 使用分支限界

● 实例分析

● 代价函数设定

● 变元排序实例

● 代价函数与分支策略

● 运行实例

● 小结

● 分支限界

● 分支限界

● 组合优化

● 代价函数

● 界

● 使用分支限界

● 实例分析

● 代价函数设定

● 变元排序实例

● 代价函数与分支策略

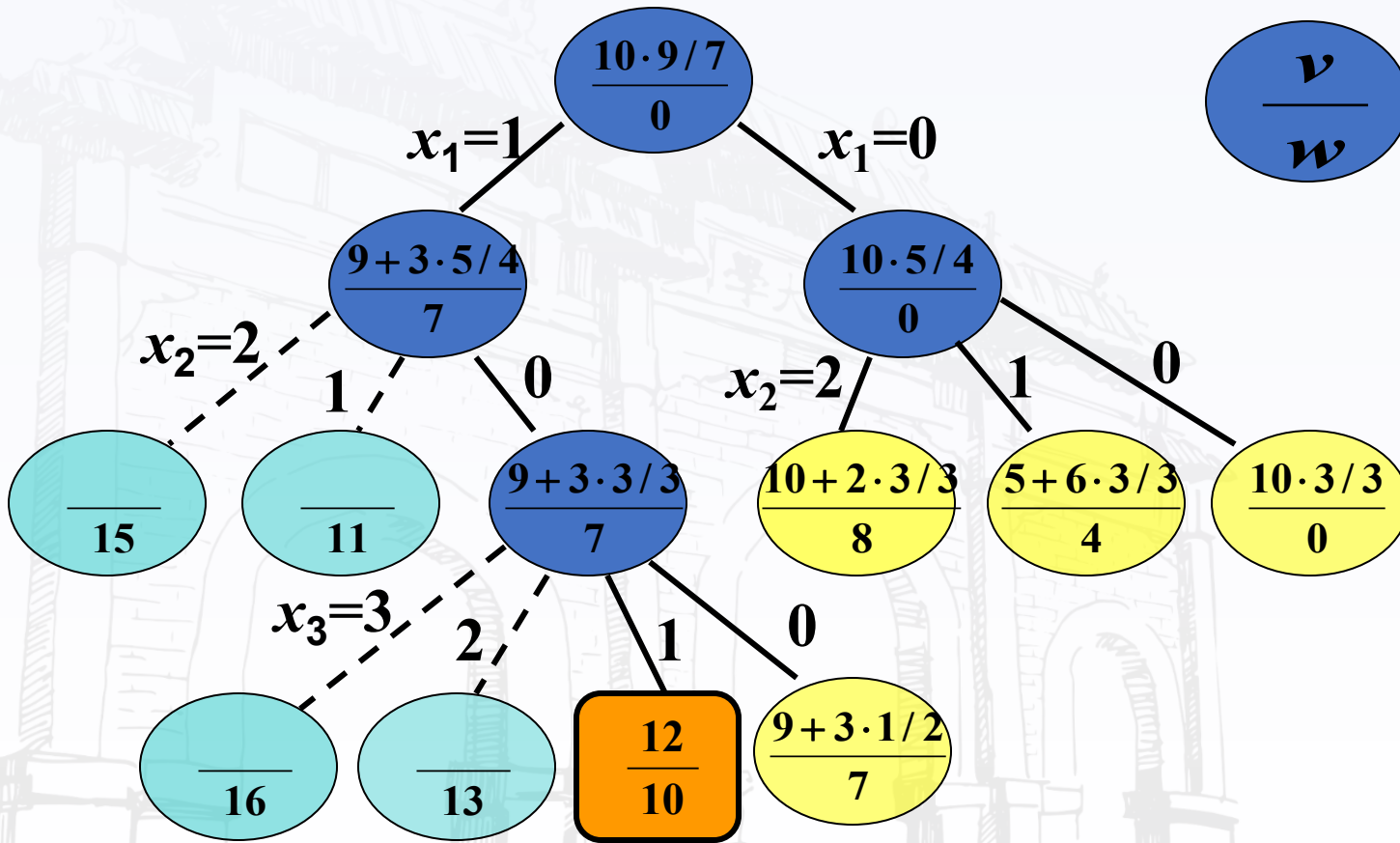
● 运行实例

● 小结

运行实例

$$\max 9x_1 + 5x_2 + 3x_3 + x_4$$

$$7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10, x_i \in \mathbb{N}, i = 1, 2, 3, 4$$



小结

分支限界适用于：组合优化问题

代价函数：

当前结点为根子树的可行解的**上界**
极大化与极小化问题不同

界：

初值为0

遇到更好可行解则更新



一、回溯算法例子

4皇后问题

0-1背包问题

货郎问题

适用条件：多米诺性质

二、回溯算法设计步骤

递归实现

迭代实现

装载问题

图着色问题

三、分支限界

一般背包问题、最大团问题

货郎问题、圆排列问题、邮资问题



● 货郎问题

● 问题定义

● 算法设计

● 代价函数与界

● 代价函数计算

● 搜索树

● 实例运行

● 算法分析

● 小结

货郎问题

问题：有 n 个城市，已知任两个城市之间的距离，求一条每个城市恰好经过1次的回路，使得总长度最小。

求： $1, 2, \dots, n$ 的排列 $k_1, k_2, \dots, k_n$ 使得

$$\min\{\sum_{i=1}^{n-1} d(c_i, c_{i+1}) + d(c_n, c_1)\}$$



● 货郎问题

● 问题定义

● 算法设计

● 代价函数与界

● 代价函数计算

● 搜索树

● 实例运行

● 算法分析

● 小结

算法设计

解向量: $\langle 1, i_1, \dots, i_{n-1} \rangle$, 其中 $i_1, \dots, i_{n-1}$ 为 $\{2, 3, \dots, n\}$ 的排列.

排列树: 整个搜索空间为一颗排列树, 结点 $\langle i_1, \dots, i_k \rangle$, 表示得到 k 步路线.

约束条件: 令 $B = \langle i_1, \dots, i_k \rangle$, 则 $i_{k+1} \in \{2, \dots, n\} - B$
即每个结点只能访问一次.



● 货郎问题

● 问题定义

● 算法设计

● 代价函数与界

● 代价函数计算

● 搜索树

● 实例运行

● 算法分析

● 小结

代价函数与界

界：当前得到的最短巡回路线长度

代价函数：设顶点 c_i 出发的最短边长度为 l_i , d_j 为选定巡回路线中第 j 段的长度, 则

$$L = \sum_{j=1}^k d_j + l_{i_k} + \sum_{i_j \notin B} l_{i_j}$$

已走过的路程 剩余长度的下界

货郎问题

问题定义

算法设计

代价函数与界

代价函数计算

搜索树

实例运行

算法分析

小结

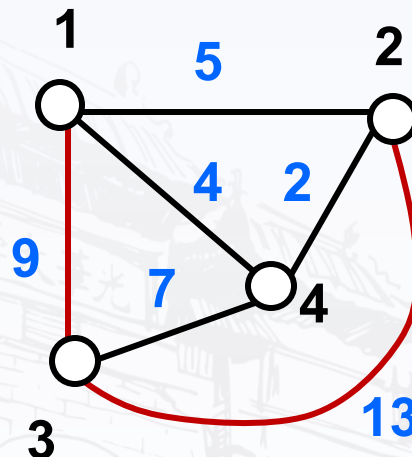
代价函数的计算

$$L = \sum_{j=1}^k d_j + l_{i_k} + \sum_{i_j \notin B} l_{i_j}$$

路线 $\langle 1, 3, 2 \rangle$ 的长度?

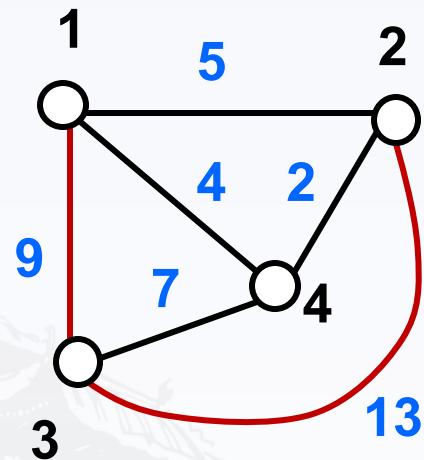
$$L = 9 + 13 + 2 + 2 = 26$$

- $9 + 13$ 是前方已经走过的路程
- $2 + 2$ 是从剩余结点2与4出发的最短边长



搜索树

-





● 货郎问题

● 问题定义

● 算法设计

● 代价函数与界

● 代价函数计算

● 搜索树

● 实例运行

● 算法分析

● 小结

实例运行

深度优先遍历搜索树

- 第一个界: $\langle 1, 2, 3, 4 \rangle$, 长度为29
- 第二个界: $\langle 1, 2, 4, 3 \rangle$, 长度为23
- 结点: $\langle 1, 3, 2 \rangle$, 代价函数值 $26 > 23$, 不再搜索, 返回 $\langle 1, 3 \rangle$, 右子树向下
- 结点 $\langle 1, 3, 4 \rangle$, 代价函数值 $9 + 7 + 2 + 2 = 20$, 继续, 可以得到可行解 $\langle 1, 3, 4, 2 \rangle$, 长度23.
- 回溯到结点 $\langle 1 \rangle$, 沿着 $\langle 1, 4 \rangle$ 向下...
- 最优解 $\langle 1, 2, 4, 3 \rangle$ 或 $\langle 1, 3, 4, 2 \rangle$, 长度23



● 货郎问题

● 问题定义

● 算法设计

● 代价函数与界

● 代价函数计算

● 搜索树

● 实例运行

● 算法分析

● 小结

算法分析

搜索树的树叶个数: $O((n-1)!)$

- 每片树叶对应1条路径, 每条路径有 n 个结点.
- 每个结点代价函数计算时间 $O(1)$, 每条路径的计算时间为 $O(n)$
- 最坏情况下算法的时间 $O(n!)$
- 实际情况下平均时间会比蛮力好很多



小结

- 货郎问题的分支限界算法：

约束条件：只能选没有走过的结点

代价函数：走过长度+后续长度的下界

- 时间复杂度 $O(n!)$ ：

● 圆排列问题

● 问题定义

● 算法设计

● 符号说明

● 长度估计

● 有关量计算

● 代价函数

● 时间复杂度

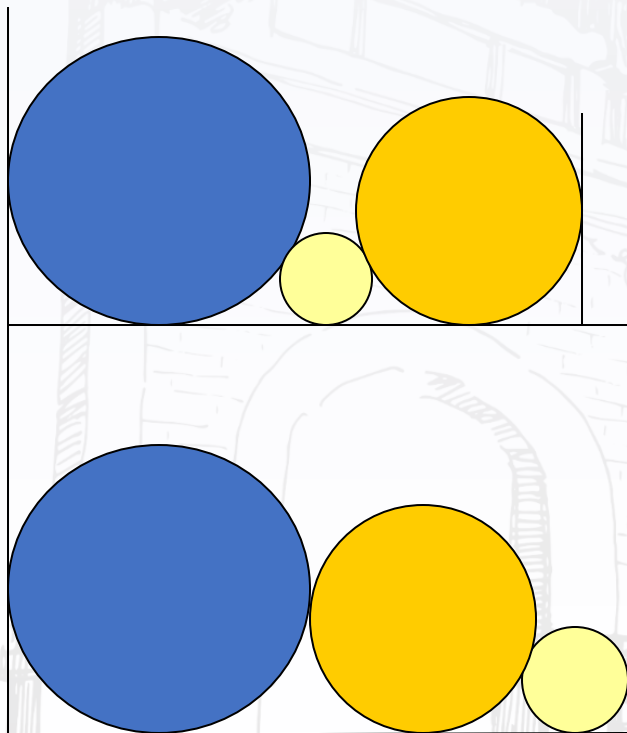
● 实例

● 实例计算

● 小结

圆排列问题

问题定义： 给定 n 个圆的半径序列, 将各圆与矩形底边相切排列, 求具有最小长度 l_n 的圆的排列顺序.



三个圆， 半径给定两种排列
具有不同排列长度，第一种
方式具有更小的排列长度.



圆排列问题

- 问题定义
- 算法设计
- 符号说明
- 长度估计
- 有关量计算
- 代价函数
- 时间复杂度
- 实例
- 实例计算
- 小结

算法设计：分支限界

解： $\langle i_1, i_2, \dots, i_n \rangle$ 为 $1, 2, \dots, n$ 的排列搜索空间为排列数

部分解向量： $\langle i_1, i_2, \dots, i_k \rangle$ 表示前 k 个圆形已排好. 令

$$B = \langle i_1, i_2, \dots, i_k \rangle$$

下一个圆选择： i_{k+1}

约束条件： $i_{k+1} \in \{1, 2, \dots, n\} - B$

界：当前得到的最小圆排列长度



圆排列问题

- 问题定义
- 算法设计
- 符号说明
- 长度估计
- 有关量计算
- 代价函数
- 时间复杂度
- 实例
- 实例计算
- 小结

符号说明

k : 算法完成第 k 步, 已经选择了第 1— k 个圆

r_k : 第 k 个圆的半径

d_k : 第 $k-1$ 个圆到第 k 个圆的圆心水平距离, $k > 1$

x_k : 第 k 个圆的圆心坐标, 规定 $x_1 = 0$,

l_k : 第 1— k 个圆的排列长度

L_k : 放好 1— k 个圆以后, 对应结点的代价函数值

$L_k \leq l_n$

圆排列问题

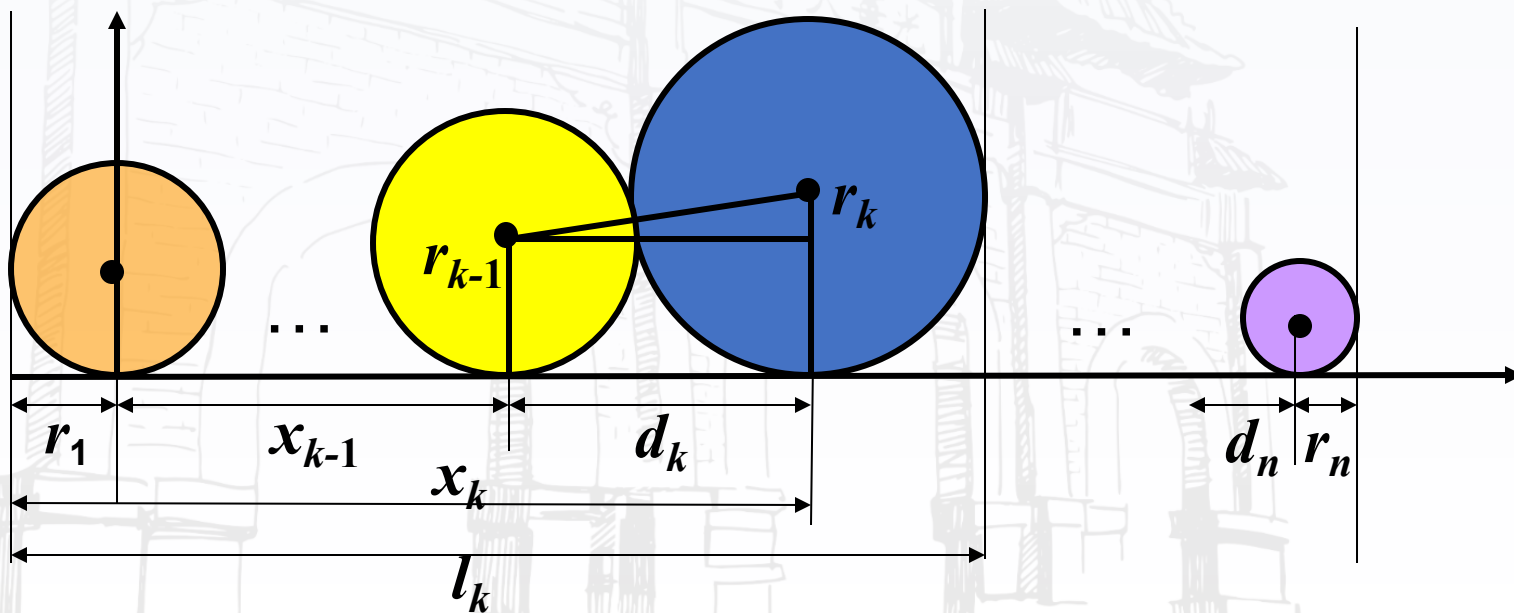
- 问题定义
- 算法设计
- 符号说明
- 长度估计
- 有关量计算
- 代价函数
- 时间复杂度
- 实例
- 实例计算
- 小结

圆排列长度估计

$$x_k = x_{k-1} + d_k$$

部分排列长度 $l_k = x_k + r_k + r_1$

排列长度 $l_n = x_k + d_{k+1} + d_{k+2} + \cdots d_n + r_n + r_1$



● 圆排列问题

● 问题定义

● 算法设计

● 符号说明

● 长度估计

● 有关量计算

● 代价函数

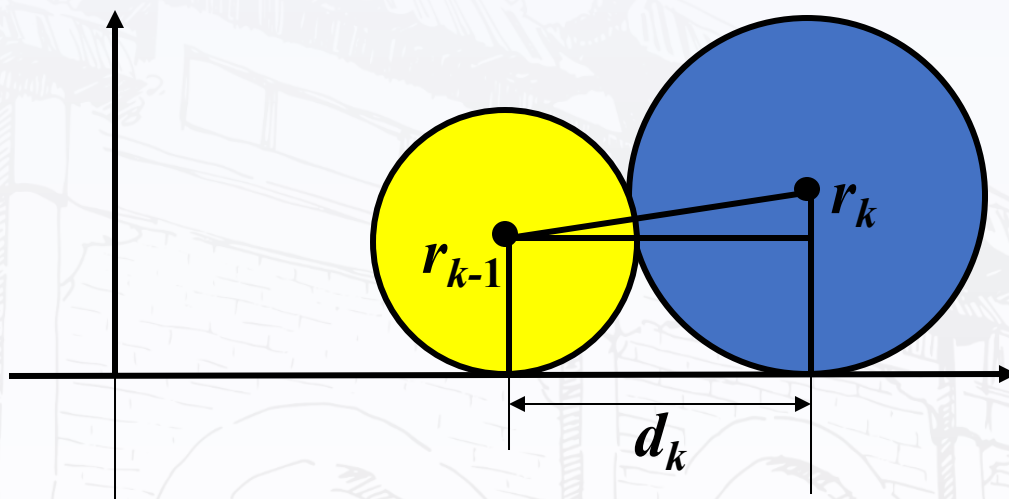
● 时间复杂度

● 实例

● 实例计算

● 小结

有关量的计算



$$\begin{aligned} d_k &= \sqrt{(r_{k-1} + r_k)^2 - (r_{k-1} - r_k)^2} \\ &= 2\sqrt{r_{k-1}r_k} \end{aligned}$$



圆排列问题

问题定义

算法设计

符号说明

长度估计

有关量计算

代价函数

时间复杂度

实例

实例计算

小结

代价函数

排列长度是 l_n , L 是代价函数:

$$l_n = x_k + 2\sqrt{r_k r_{k+1}} + 2\sqrt{r_{k+1} r_{k+2}} + \dots + 2\sqrt{r_{n-1} r_n} + r_n + r_1$$
$$\geq x_k + 2(n-k)r + r + r_1$$

$$L = x_k + (2n - 2k + 1)r + r_1$$

$$r = \min(r_{i_j}, r_k) \quad i_j \in \{1, 2, \dots, n\} - B$$

$$B = \{i_1, i_2, \dots, i_k\},$$



圆排列问题

- 问题定义
- 算法设计
- 符号说明
- 长度估计
- 有关量计算
- 代价函数
- 时间复杂度
- 实例
- 实例计算
- 小结

时间复杂度

- 搜索树的树叶为 $O(n!)$
- 每条路径代价函数的计算为 $O(n)$
- 最坏情况下算法时间复杂度为

$$O(nn!) = O((n+1)!)$$

圆排列问题

问题定义

算法设计

符号说明

长度估计

有关量计算

代价函数

时间复杂度

实例

实例计算

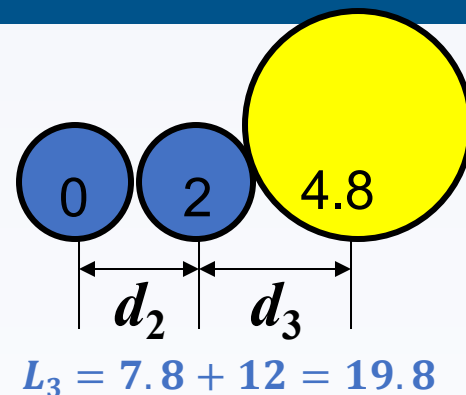
小结

实例

输入: $R = \{ 1, 1, 2, 2, 3, 5 \}$

取排列 $\langle 1, 2, 3, 4, 5, 6 \rangle$,

半径排列为: $1, 1, 2, 2, 3, 5$



k	r_k	d_k	x_k	l_k	L_k
1	1	0	0	2	12
2	1	2	2	4	12
3	2	2.8	4.8	7.8	19.8
4	2	4	8.8	11.8	19.8
5	3	4.9	13.7	17.7	23.7
6	5	7.7	21.4	27.4	27.4



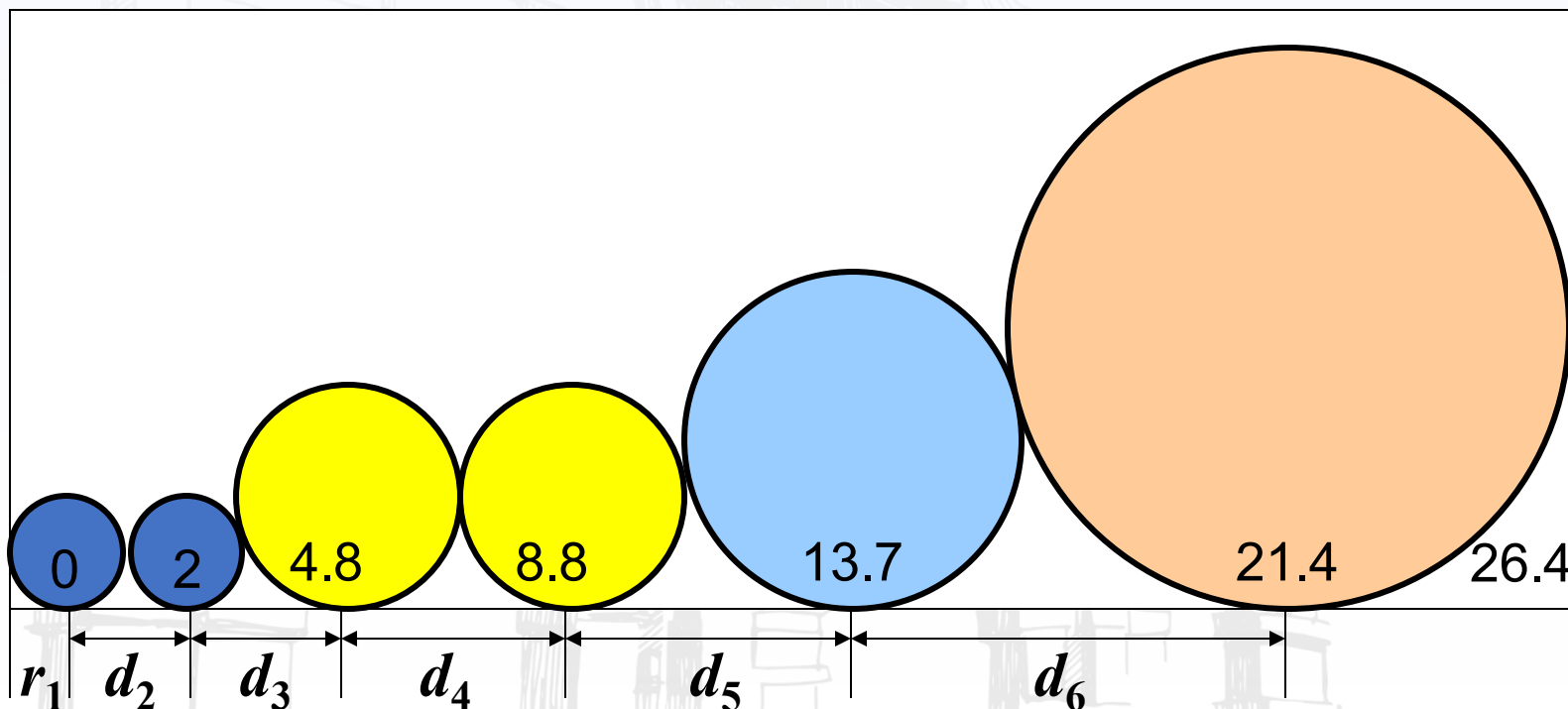
圆排列问题

- 问题定义
- 算法设计
- 符号说明
- 长度估计
- 有关量计算
- 代价函数
- 时间复杂度
- 实例
- 实例计算
- 小结

实例计算

$$R = \{1, 1, 2, 2, 3, 5\}$$

取排列 $\langle 1, 2, 3, 4, 5, 6 \rangle$, 半径排列为: 1, 1, 2, 2, 3, 5,
最短长度 $l_6 = 27.4$ 最优解 $\langle 1, 3, 5, 6, 4, 2 \rangle$, 26.5





小结

- 圆排列问题的定义
- 如何设计代价函数？
 - 已有的排列长度 + 后续的圆排列长度的下界
- 时间复杂度 $O((n + 1)!)$:

连续邮资问题

问题定义

算法设计

r_i 的计算

实例

小结

连续邮资问题

例5.12 连续邮资问题：给定 n 种不同面值的邮票，每个信封至多贴 m 张，试给出邮票的最佳设计，使得从1开始，增量为1的连续邮资区间达到最大？

实例： $n = 5, m = 4,$

面值 $X_1 = \langle 1, 3, 11, 15, 32 \rangle,$

邮资连续区间为 $\{1, 2, \dots, 70\}$

面值 $X_2 = \langle 1, 6, 10, 20, 30 \rangle,$

邮资连续区间为 $\{1, 2, 3, 4\}$





连续邮资问题

问题定义

算法设计

r_i 的计算

实例

小结

算法设计

可行解 : $\langle x_1, x_2, \dots, x_n \rangle$, $x_1 = 1$,
 $x_1 < x_2 < \dots < x_n$

搜索策略: 深度优先

约束条件: 在结点 $\langle x_1, x_2, \dots, x_i \rangle$ 处, 邮资最大连续
区间为 $\{1, \dots, r_i\}$, x_{i+1} 的取值范围是
 $\{x_i + 1, \dots, r_i + 1\}$

若 $x_{i+1} > r_i + 1$, $r_i + 1$ 的邮资将没法支付



连续邮资问题

- 问题定义
- 算法设计
- r_i 的计算
- 实例
- 小结

r_i 的计算

$y_i(j)$: 用至多 m 张面值 $x_1, x_2, \dots, x_{i-1}, x_i$ 的邮票贴 j 邮资时的最少邮票数, 则

$$y_i(j) = \min_{1 \leq t \leq m} \{t + y_{i-1}(j - tx_i)\}$$

$$y_1(j) = j$$

$$r_i = \min\{j \mid y_i(j) \leq m, y_i(j+1) > m\}$$

搜索策略: 深度优先

界: $\max, m$ 张邮票可付的连续区间的最大邮资

连续邮资问题

- 解:** $X = \langle 1, 4, 7, 8 \rangle$, 最大连续区间为 $\{1, 2, \dots, 24\}$



小结

- 圆排列问题的定义
- 如何设计代价函数？
 - 已有的排列长度 + 后续的圆排列长度的下界
- 时间复杂度 $O((n + 1)!)$:

最大团问题

问题定义

独立集与团

应用

编码设计

计算复杂度

算法设计

实例

运行过程

小结

最大团问题

最大团问题： 给定无向图 $G = \langle V, E \rangle$, 求 G 中的最大团.

相关知识：



无向图 $G = \langle V, E \rangle$,



G 的子图： $G' = \langle V', E' \rangle$, 其中 $V' \subseteq V, E' \subseteq E$,



G 的补图： $\bar{G} = \langle V, E' \rangle$, E' 是 E 关于完全图边集的补集



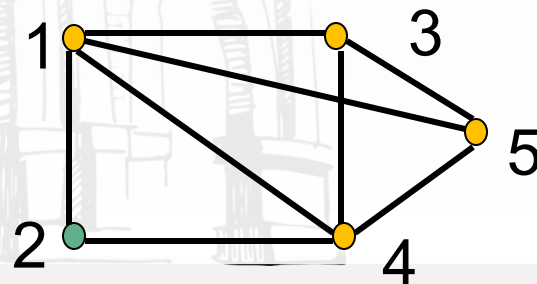
G 中的团： G 的完全子图



G 的最大团： 团中顶点数最多的团

实例

最大团： $\{1, 3, 4, 5\}$



● 最大团问题

● 问题定义

● 独立集与团

● 应用

● 编码设计

● 计算复杂度

● 算法设计

● 实例

● 运行过程

● 小结

独立集与团

G 的点**独立集**: G 的顶点子集 A , 且 $\forall u, v \in A$, 边 $(u, v) \notin E$.

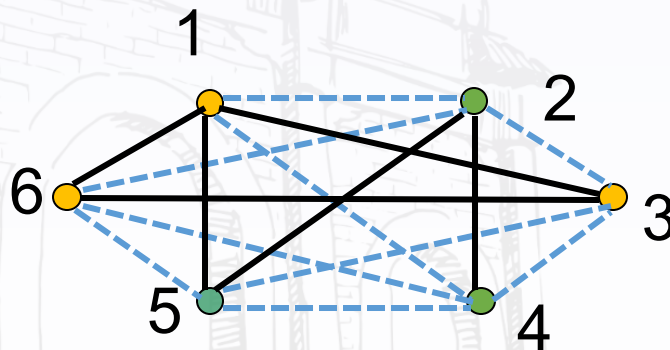
最大点独立集: 顶点数最多的点独立集

命题: U 是 G 的最大团当且仅当 U 是 $\bar{G}$ (补图) 的最大点独立集.

求 G 的最大团:

$U = \{1, 3, 6\}$

等价于求补图 $\bar{G}$ 的最大独立集



● 最大团问题

应用

编码, 故障诊断, 计算机视觉, 聚类分析, 经济学, 移动通讯, VLSI 电路设计

例子, 噪音使信道传输字符发生混淆

混淆图 $G = \langle V, E \rangle$, V 为有穷字符集合, $\{u, v\} \in E \iff u$ 和 v 易混淆



● 最大团问题

● 问题定义

● 独立集与团

● 应用

● 编码设计

● 计算复杂度

● 算法设计

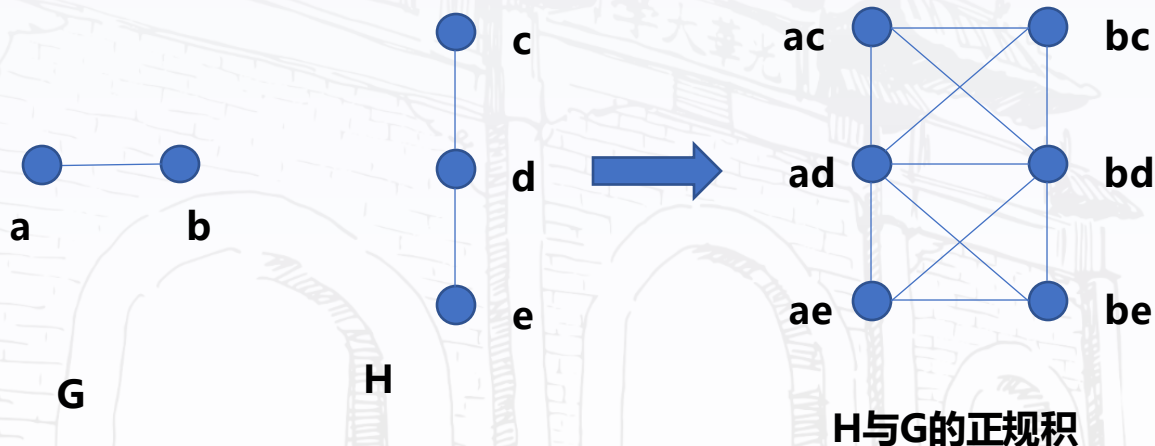
● 实例

● 运行过程

● 小结

应用:编码设计

xy 与 uv 混淆 $\Leftrightarrow x$ 与 u 混淆且 y 与 v 混淆 $\vee$
 $x=u$ 且 $y=v$ 混淆 $\vee x$ 与 u 混淆且 $y=v$



为减少噪音干扰，设计代码应该找混淆图中的最大点独立集



● 最大团问题

● 问题定义

● 独立集与团

● 应用

● 编码设计

● 计算复杂度

● 算法设计

● 实例

● 运行过程

● 小结

最大团计算复杂度

最大团问题：给定无向图 $G = \langle V, E \rangle$ ，求 G 中的最大团。

解： $\langle x_1, x_2, \dots, x_n \rangle$ 为 0-1 向量， $x_k = 1$ 当且仅当顶点 k 属于最大团。

蛮力算法： 对每个顶点集，检查是否构成团，即其中每对顶点之间是否都有边。有 2^n 个子集，至少需要指数时间。



● 最大团问题

● 问题定义

● 独立集与团

● 应用

● 编码设计

● 计算复杂度

● 算法设计

● 实例

● 运行过程

● 小结

算法设计

搜索树为子集树，结点 $\langle x_1, x_2, \dots, x_k \rangle$ 的含义：

已检索 k 个顶点，其中 $x_i = 1$ 对应的顶点在当前的团内
搜索树为子集树

约束条件：分支处顶点与当前团内每个顶点都有边相连

界：当前图中已检索到的极大团的顶点数

代价函数：目前的团扩张为极大团的顶点数上界

$$F = C_n + n - k$$

其中 C_n 为目前团的顶点数（初始为 0），

k 为结点层数

时间： $O(n2^n)$

● 最大团问题

● 问题定义

● 独立集与团

● 应用

● 编码设计

● 计算复杂度

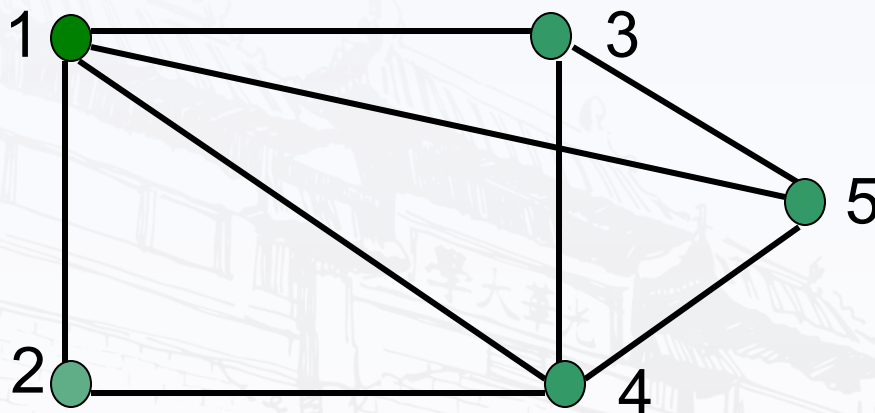
● 算法设计

● 实例

● 运行过程

● 小结

最大团实例



顶点编号顺序为 1, 2, 3, 4, 5,
对应 x_1, x_2, x_3, x_4, x_5 , $x_i=1$ 当且仅当 i 在团内
分支规定左子树为 1, 右子树为 0.
 B 为界, F 为代价函数值.



小结

- 适于求解组合搜索问题及优化问题？
- 求解条件：满足多米诺性质
- 解的表示：解向量，求解是不断扩充解的过程
- 回溯条件：
 - 搜索问题-约束条件 优化问题-约束条件+代价函数
- 分支策略：深度优先、宽度优先、多重优先
- 时间复杂度： $W(n) = p(n)f(n)$
 - $P(n)$ 为每个结点的工作量， $f(n)$ 为结点个数
 - 最坏情况下时间通常为指数级别、平均情况下比蛮力算法好、空间代价小
- 降低时间复杂性的主要途径：
 - 根据树的分支设计优先策略：结点少分枝优先，解多分枝优先
 - 利用对成性裁剪子树

算法课程的知识框架

分治策略

动态规划

贪心算法

回溯与分治限界

主要步骤
分析方法
典型例子

算法的数学基础

序列求和

递推方程求解

算法的基本概念

算法的伪码
表示

算法的两种
复杂度

复杂度阶的
表示

函数的阶

- 阶的符号: O o Θ Ω ω

- 阶的高低:

常数
线性函数
对数
二次函数
三次函数
指数
阶乘



$$2^{2^n}, \quad n!, \quad n2^n, \quad (3/2)^n, \quad (\log n)^{\log n} = n^{\log \log n},$$

$$n^3, \quad \log(n!) = \Theta(n \log n), \quad n = \Theta(2^{\log n}),$$

$$\log^2 n, \quad \log n, \quad \sqrt{\log n}, \quad \log \log n,$$

$$n^{1/\log n} = \Theta(1)$$



序列求和

• 基本求和公式

等比数列

$$\sum_{k=1}^n a_k = \frac{n(a_1 + a_n)}{2}$$

等差数列

$$\sum_{k=0}^n aq^k = \frac{a(1-q^{n+1})}{1-q}, \quad \sum_{k=0}^n x^k = \frac{1-x^{n+1}}{1-x}$$

调和级数

$$\sum_{k=1}^n \frac{1}{k} = \ln n + O(1)$$

• 估计和式的阶:

放大法



估计上界

积分法



估计上下界

递推方程求解

- 主要求解方法

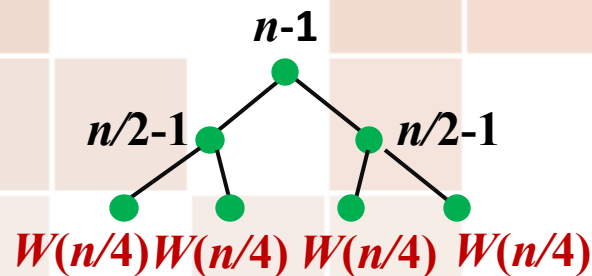
迭代+进行序列求和

递归树+求和

主定理

- 常见递推方程

$$T(n) = aT(n/b) + d(n)$$



算法设计技术

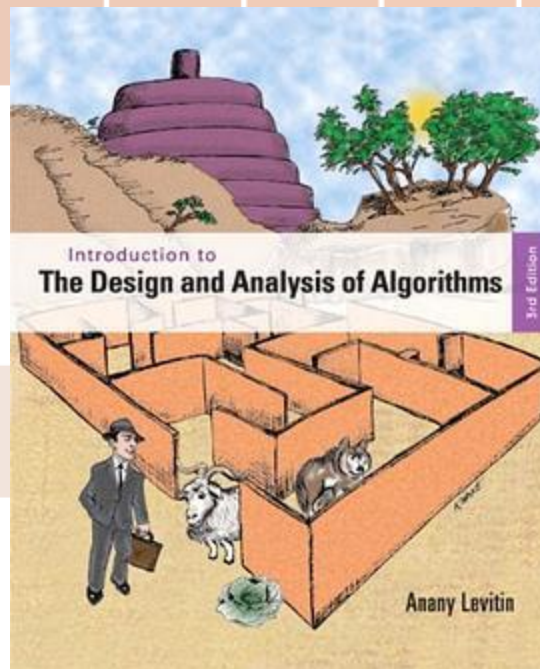
- 设计技术

分治策略

动态规划

贪心算法

回溯和分支限界



参考《The Design and Analysis of Algorithms》Anany Levitin

降治策略 Decrease & Conquer

变治策略 Transform & Conquer

- 关注的问题

步骤、条件、复杂度
典型例子、改进方法



分治策略

- **适用条件：**规约为独立求解子问题

- **设计步骤：**

分解

解决

合并

***注意要划分均衡、类型相同**

- **递归算法分析：**求解递推方程 $T(n) = aT(\frac{n}{b}) + f(n)$

- **改进的途径：**减少子问题，增加预处理

- **典型问题：**二分检索 归并排序 芯片测试 矩阵乘法 临近点对



动态规划

- **适用条件：** 优化问题、多步判断求解、原问题最优性原则、子问题重叠

- **设计步骤：**

子问题边界

递推方程
及初值

自底向上

备忘录

解追踪

- **复杂度分析：** 备忘录、递推方程

- **典型问题：**

矩阵链相乘

投资问题

背包问题

最大子段和

图像压缩

贪心法

- **适用条件：** 组合优化问题、多步判断、贪心（短视）选择性质

- **设计步骤：**

局部优化
策略

证明方法

反正

直接证明

数学归纳

- **复杂度分析：** $O(m^n)$ or $O(nm^n)$

- **典型问题：** 活动选择 装载问题 最小延迟调度 最优前缀码 最小生成树



回溯和分支限界

- **适用条件：** 搜索或优化问题、多步判断求解

- **设计步骤：**

确定解向量

确定搜索
树

确定搜索
顺序

搜索约束条件
与代价函数

路径存储

- **复杂度分析：** 搜索树结点数，最坏情况与蛮力算法相当，但是平均复杂度下降

- **典型问题：**

N后问题

背包问题

装载问题

圆排列问题

货郎问题

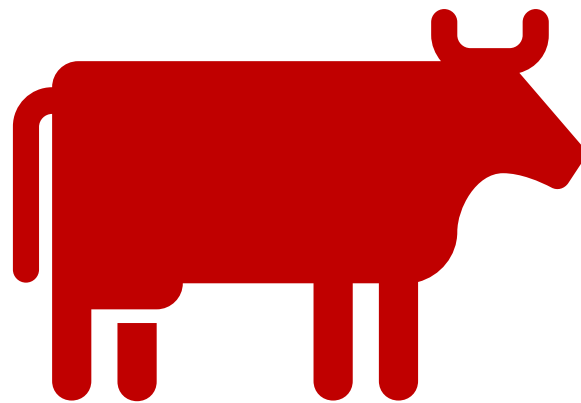
算法设计

- **设计思想**：尽量选复杂度低的算法
- **数据结构**：算法实现依赖于合适的数据结构
- **时空权衡**：实际问题中需要权衡实现时间、空间与成本



谢谢大家！

祝大家学业有成！



都能成为人工智能领域大牛！